Ayyoub CHETOUANI
BTS SIO
Lycée Saint Rémi - Site Léonard De Vinci
Année scolaire 2025-2026

Synthèse de mon Stage à l'entreprise Logic@l Conseils

Stage du 5 janvier au 20 février

Responsable professionnel : Jean-Phillipe Fontaine
Responsable scolaire : Sylvain Guillemin
Professeur principal : Nathalie Bâcle

SOMMAIRE

Remerciements

Présentation de l'entreprise Logic@l Conseils

Réalisation Professionnelle

Remerciements

J'adresse mes remerciements à l'entreprise Logic@l Conseils, notamment au PDG Monsieur HAMDANI Malik et l'équipe Logic@l pour leur accueil et leurs conseils tout au long du stage.

J'aimerais aussi adresser mes remerciements à mon maître de stage Monsieur FONTAINE Jean-Philippe pour ses enseignements et sa patience durant la période de stage, mais également à l'équipe de pôle Recherche et Développement avec qui j'ai travaillé et qui ont su me guider tout au long de la réalisation professionnelle.

Présentation de l'entreprise Logic@l Conseils

Qu'est-ce que Logic@l Conseils ?

Logic@l Conseils est une ESN ou entreprise de services numérique, il s'agit d'une entreprise spécialisée dans la prestation informatique pour les entreprises. En d'autres termes son rôle est d'accompagner les entreprises clientes à travers une évolution technologique par le déploiement de consultant directement chez ses clients afin de répondre à leurs besoins spécifiques en matière de technologie.

Les différents services de Logic@l Conseils

Il existe plusieurs services dans cette entreprise, on peut distinguer :

- Le service RH qui gère l'administration, la partie comptabilité et la vie en entreprise comme gérer les congés payés ou répondre aux questions des consultants sur des sujets liés à l'entreprise et son fonctionnement.
- Le service Recrutement qui gère le recrutement des consultants au sein de la boîte, en cherchant des clients sur des plateformes de recrutement ou en faisant des déplacements dans des lieux cibles comme des écoles ou d'autres endroits, à la suite de ceci, il fait passer des entretiens aux candidats pour savoir si ce dernier correspond à ce dont le client a besoin.
- Le service communication qui est chargé d'effectuer la communication en interne et en externe vis à vis de l'entreprise, sous plusieurs formats comme des articles, des posts sur les réseaux sociaux, etc....
- Le service commercial qui est chargé de persuader le client de louer un ou plusieurs consultants pour son besoin

Pour ma part, j'ai travaillé auprès du pôle Recherche et Développement qui est une structure de l'entreprise qui a la charge de développer des technologies internes à l'entreprise et qui travaille sur les technologies les plus innovantes afin de permettre à l'entreprise de toujours se situer à la pointe de l'innovation et d'être un accélérateur pour les projets à la fois interne et clients. C'est un pôle qui est donc transverse au sein de Logical Conseils.

Les clients

Il existe de nombreux clients liés à Logic@l Conseils, on peut notamment citer :

- Auchan Retail
- Adeo
- AG2R LA MONDIALE
- AXA Banque
- Bonduelle
- Boulanger
- Crédit Agricole Consumer Finance
- Decathlon
- EUROTUNNEL HOLDING
- Kiloutou
- Leroy Merlin
- ÏDKIDS
- KIABI
- …

Organigramme de l'entreprise

Figure 1 : Organigramme de Logical Conseils

Les partenaires

Il existe de nombreux partenaires liés à Logic@l Conseils, on peut notamment citer :

- Stambia
- Informatica
- Semarchy
- Snowflake

Les domaines d'activités

Logic@l Conseils est spécialisée dans 3 domaines d'activités différents, en particulier sur :

- **Software Engineering** : pôle se concentrant sur les langages de programmation et Frameworks d'aujourd'hui et de demain comme l'IA agentique qui est en plein essor, mais aussi sur des technologies plus anciennes comme par exemple SAP HANA. Les architectes de ce pôle sont principalement orientés vers le back-end avec Python et Java, avec quelques notes de front-end grâce à Javascript.

- **Data Analytics :** pôle se concentrant sur l'analyse, plus précisément sur ces 4 points :
  - **Analyse de l'existant** : Définir des métriques pertinentes partagées par l'entreprise. Identifier les sources de données fiables.

  - **Plans et règles de diffusion** : Transmettre les données selon les règles de diffusion. Sécuriser et fiabiliser les données.

  - **Outillage** : Consolider les indicateurs d'un Data Warehouse et les diffuser. Choisir la de Reporting et de gestion de flux de données.

  - **Actions du management :** Aboutir à une qualité du suivi du business accrue. Conforter la prise de décision du client.

- **Data Management** : pôle se concentrant sur la gestion et la sauvegarde des données, notamment le patrimoine informatique de l'entreprise via ces méthodes :
  - **Data management :** ensemble de processus, d'outils et de techniques visant à exploiter les données de manière efficace. Il assure la sécurité, la cohérence et la qualité des données, ce qui est crucial pour les entreprises de tous les secteurs. Les données sont devenues un atout stratégique majeur, utilisées pour réduire les coûts, optimiser les processus de gestion et améliorer les campagnes marketing.

- **Data Quality** : essentielle pour le succès à long terme des stratégies d'une entreprise. Elle désigne la capacité d'une entreprise à maintenir ses données précises et utiles au fil du temps. Avec le temps, certaines données peuvent devenir obsolètes ou incorrectes, perdant ainsi leur valeur et leur utilité. Par exemple, dans une base de données contenant des adresses e-mail de clients, certaines adresses peuvent devenir invalides si les clients changent d'adresse email ou cessent de les utiliser. Il est donc crucial de nettoyer régulièrement la base de données pour s'assurer que les informations ne contiennent pas de valeurs aberrantes ou manquantes, dans le cas contraire cela peut mettre en doute la fiabilité de l'algorithme.

- **GDPR/RGPD** : Le RGPD ne concerne que les données personnelles et non celles des personnes morales.

- **Data Governance**: essentielle pour garantir le contrôle, la qualité et la sécurité des données au sein d'une entreprise. Les incohérences dans les données peuvent avoir des conséquences graves sur leur intégration et leur intégrité, affectant ainsi la précision de la Business Intelligence, du reporting d'entreprise et des analyses. Pour éviter ces problèmes, il est crucial d'instaurer une gouvernance des données qui consiste avant tout à assurer un contrôle sur qui a accès à quoi et comment. On définit des rôles utilisateurs (politique RBAC) afin de donner des droits granulaires à certaines données. On fait du lineage de données aussi. C'est à dire qu'avec un outil de gouvernance on est capable de tracer la provenance d'une donnée (table, colonne, schéma etc…) de manière très précise.

Voici les principes fondamentaux que les entreprises doivent respecter selon le RGPD :

- **Licéité, loyauté et transparence** (Article 5 :1-a) (RGPD, s.d.)*:* Les entreprises doivent traiter les données de manière **licite**, **loyale** et **transparente**. Elles sont tenues d'informer clairement les personnes concernées sur la collecte, l'usage et la finalité de leurs données.

- **Limitation des finalités** (Article 5 : 1-b) (RGPD, s.d.)*:* Les données doivent être collectées pour des **objectifs précis, explicites et légitimes**. Elles ne peuvent pas être réutilisées pour d'autres finalités incompatibles avec celles annoncées initialement.

- Exactitude, conservation limitée et sécurité :

  - **Exactitude** *(Article 5 :1-d)* (RGPD, s.d.): les données doivent être exactes et mises à jour.

  - **Limitation de la conservation** *(Article 5 :1-e* (RGPD, s.d.): les données ne doivent pas être conservées plus longtemps que nécessaire.

- **Intégrité et confidentialité** *(Article 5 :1-f)* (RGPD, s.d.) : les entreprises doivent garantir la sécurité des données, notamment contre l'accès non autorisé, la perte ou l'altération.

– **Minimisation des données** *(Article 5-1-c)* (RGPD, s.d.): Les entreprises ne doivent collecter que les données **strictement nécessaires** au regard de la finalité du traitement. Toute information superflue doit être exclue.

– **Responsabilités** (RGPD, s.d.) (RGPD, s.d.) (RGPD, s.d.): Les entreprises sont responsables du respect de l'ensemble de ces principes et doivent être en mesure de démontrer leur conformité. Cela inclut, lorsque nécessaire, la désignation d'un Délégué à la Protection des Données (DPO) (Articles 37 à 39).

- **Pseudonymisation** : également connue sous le nom d'anonymisation réversible, est une technique utilisée pour modifier certains attributs dans une base de données afin de protéger les informations personnelles. Cette méthode permet de traiter les données de manière qu'elles ne puissent pas être directement liées à une personne sans l'utilisation d'informations supplémentaires. Cela limite la corrélation entre différentes informations nominatives, augmentant ainsi la sécurité des données personnelles.

  Cette technique est souvent employée pour sécuriser les informations personnelles dans les entreprises. Par exemple, une entreprise peut souhaiter limiter l'accès aux informations personnelles de ses clients à certains employés. Dans ce cas, la pseudonymisation peut être utilisée pour remplacer certains attributs sensibles par d'autres valeurs, tout en sécurisant les attributs originaux dans un endroit inaccessible à ces employés.

Avantages de la Pseudonymisation

- **Conformité avec le RGPD** : La pseudonymisation est recommandée par le Règlement Général sur la Protection des Données (RGPD). Le RGPD est plus souple avec les données pseudonymisées par rapport aux données personnelles non traitées, ce qui en fait une technique avantageuse pour le traitement des données

- **Réduction des risques de violation** : En cas de violation de données pseudonymisées, l'impact sur les personnes concernées est moindre par rapport à une violation de données personnelles non protégées. Cela permet à l'entreprise d'éviter des poursuites judiciaires potentielles qui pourraient être engagées par les personnes affectées qui peuvent aller jusqu'à des amendes dont le montant correspond à 2% du chiffre d'affaires annuel de l'entreprise (RGPD, s.d.).

- **Profiling** : technique essentielle pour garantir la qualité des données au sein d'une entreprise. La qualité des données est directement liée à la manière dont elles sont profilées, et une mauvaise qualité peut entraîner des pertes financières importantes pour les entreprises, car les décisions marketing et commerciales sont souvent basées sur ces données. Le profilage permet d'analyser et de traiter les données pour détecter et corriger les problèmes, améliorant ainsi leur qualité et leur utilité pour l'entreprise.

  Le profilage des données consiste à examiner minutieusement les données pour déterminer leur légitimité et leur qualité. Les algorithmes utilisés évaluent les données selon divers critères tels que le percentile, la fréquence, le maximum et le minimum, et analysent les métadonnées pour identifier les erreurs récurrentes et coûteuses dans les bases de données. Cette opération peut être intégrée dans les tâches quotidiennes des spécialistes des données ou être sous-traitée à une entreprise de services numériques.

Comment Logic@l Conseils gère son patrimoine informatique ?

Logic@l Conseils gère son patrimoine informatique de plusieurs manières :

- Au niveau matériel, l'entreprise gère le matériel informatique grâce à l'un de ces partenaires : ADVCOM.
  Cette entreprise fournit les ordinateurs pour les consultants et les configure en fonction de ce dont l'entreprise a besoin, tout en configurant des adresses professionnelles pour l'utilisation des services tel que Microsoft.

- Au niveau logiciel, la plupart des logiciels sont open source comme Visual Studio Code ou Git ou bien achetés par l'entreprise comme la suite Microsoft, mais parfois dans le cadre de mission pour des entreprises clientes, l'entreprise qui est démarchée peut donner les accès à un logiciel à un ou plusieurs consultants pour qu'ils mènent à bien la mission.

- Au niveau personnel, afin que les employés de n'importe quel service puissent utiliser à bien les logiciels fournis, que ça soit lors de mission ou en général, ils peuvent être amenés à effectuer une formation plus au moins grande maîtriser les fondements du logiciel, exemple pour la suite Microsoft, pour ceux travaillant dans l'administration ou ailleurs

Comment Logic@l Conseils effectue sa veille technologique ?

Etant également un rôle du Pole Recherche & Développement, la veille technologique peut se faire via l'intermédiaire de ce Pôle qui présentera les dernières nouveautés dans le domaine de la data et de l'IA, de prototyper des technologies et de transformer des idées en projets concrets.

Le tout appuyé par le pôle de Communication qui se chargera de le partager aux autres consultants.

Les consultants de Logic@l Conseils effectuent leur veille technologique via plusieurs méthodes :

- Des formations payées par l'entreprise
- Des certifications
- De l'information récoltée par des recherches personnelles
- Des ateliers animés par des consultant de l'entreprise sur un thème précis

Réalisation Professionnelle

Projet : Open Data

Contexte de la mission :

Afin de faciliter l'accès aux clients de l'entreprise à des données libre d'accès venant d'API autour de différents domaines comme la météo ou bien le cours des matières premières, le but est de créer une application utilisant une API regroupant plusieurs données d'autres API qui récoltera les données avec les mises à jour de manière continuelle et en temps réel.

On m'a donné comme missions au fil de la réalisation de chercher des données liées à plusieurs, de les évaluer au niveau de la pertinence, de les récolter et de les afficher

Schéma montrant le parcours du projet durant les principales étapes :

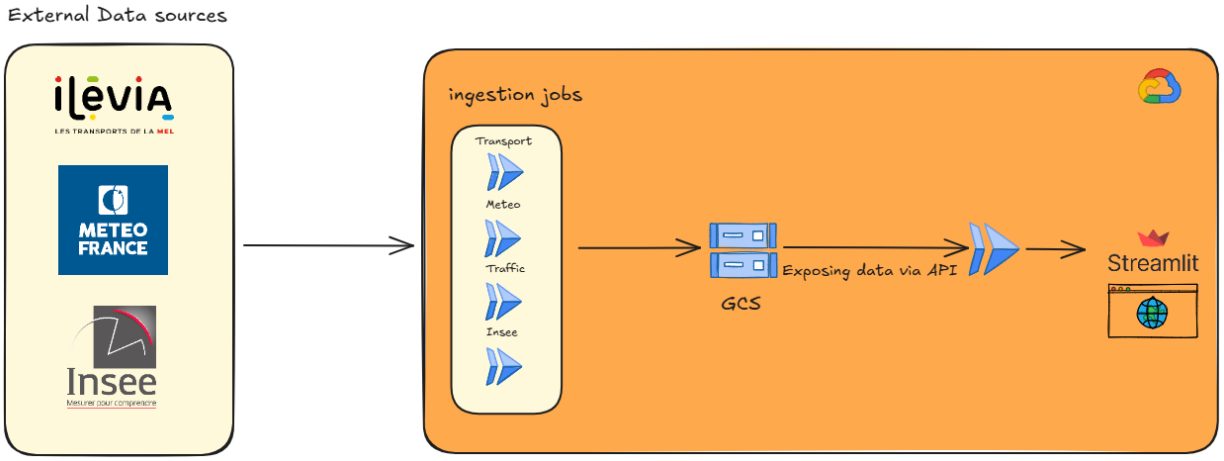


Figure 2 : Schéma du Projet Open Data

Dans le cadre de la mission j'ai pu découvrir et effectuer toutes les différentes parties, de la recherche des données jusqu'à la conception de l'application et les essais qui vont avec.

Etapes de conception de l'application :

Tout d'abord, il m'a fallu chercher les différentes API correspondantes aux données recherchées comme les données liées à l'inflation, aux matières premières ou encore au cinéma. Pour cela j'ai dû rechercher via plusieurs moyens (recherche google, aide de recherche via Intelligence Artificielle, site du gouvernement data.gouv ...) afin de trouver les bonnes données.

Une API est un intermédiaire qui permet à deux logiciels de communiquer entre eux sans que l'on ne puisse voir ce qu'il se passe.

Elle peut servir à :

- Demander des informations,
- Envoyer des données,
- Faire interagir deux systèmes qui ne se connaissent pas.

Puis après avoir évalué la pertinence des données vis-à-vis du besoin du projet et au niveau du client, on prend le lien de l'API et on effectue des tests de requête via Postman.

Postman est une application (web et desktop) permettant à l'utilisateur :

- D'envoyer des requêtes HTTP (GET, POST, PUT, DELETE, etc.) vers une API
- De voir les réponses de manière lisible sous plusieurs formats (JSON, XML, HTML...)
- Automatiser des tests (tests unitaires d'API)

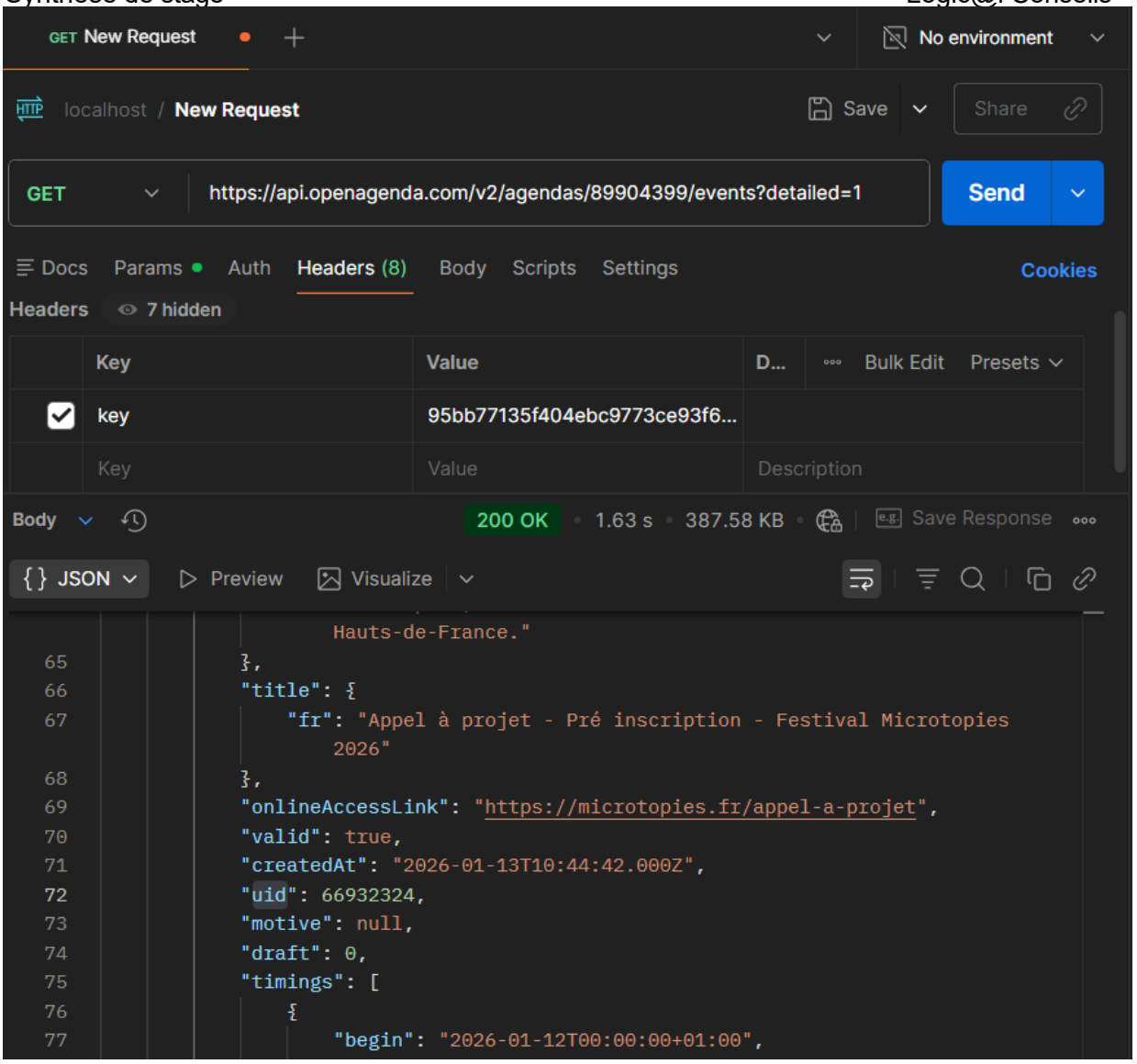


Figure 3: Capture d'écran de Postman lors d'une réponse à une requête HTTP

Ici on effectue un test afin de voir ce que propose l'API en termes de données et ce que l'on pourrait exploiter à l'intérieur

Pour donner suite à ces différents tests, on créer un script Python pouvant récupérer les données sous un format JSON avec une structure de répertoires qui permettra aux autres étapes du projet de se fluidifier plus facilement

```python
import os
import requests
import pprint
import time
import json

DOSSIER_RACINE = "Banque_France_API"


# Création des dossiers UNE SEULE FOIS
chemin = os.path.join(DOSSIER_RACINE)
os.makedirs(chemin, exist_ok=True)

# Récupération des informations pour plusieurs zones géographiques

url= "https://webstat.banque-france.fr/api/explore/v2.1/catalog/datasets/observations/exports/json/?where=series_key+IN+%28%22ICP.M

response = requests.get(url)

if response.status_code == 200:
    try:
        data_list = response.json()  # data_list est une liste
        # if isinstance(data_list, list) and len(data_list) > 0:
        #     data = data_list[0]  # prendre le premier dictionnaire
        # else:
        #     data = {}

        for data in data_list:
            # observations_attributes_and_values est une chaîne JSON
            obs_raw = data.get("observations_attributes_and_values", "{}")
            obs_dict = json.loads(obs_raw)  # convertir en dictionnaire

            obs_conf = obs_dict.get("OBS_CONF", "")
            obs_status = obs_dict.get("OBS_STATUS", "")

            data_json = {
                "DATASET_ID": data.get("dataset_id", ""),
```

Figure 4: Première partie du code : récupération des données + début du traitement

```python
            data_json = {
                "DATASET_ID": data.get("dataset_id", ""),
                "CLE_SERIE": data.get("series_key", ""),
                "TITRE": data.get("title_fr", ""),
                "PERIODE": data.get("time_period", ""),
                "PERIODE_DEBUT": data.get("time_period_start", ""),
                "PERIODE_FIN": data.get("time_period_end", ""),
                "VALEUR": data.get("obs_value", ""),
                "STATUT": data.get("obs_status", ""),
                "PUBLICATION_DATE": data.get("updated_at", ""),
                "OBSEVATION": {
                    "VALEUR": obs_conf,
                    "STATUT": obs_status,
                },
                "BASKETS": data.get("baskets", []),
                "GREATEST_UPDATED_AT": data.get("greatest_updated_at", ""),
                "SERIE_UPLOAD_AT": data.get("series_greatest_updated_at", ""),
                "LAST_THEMES_UPDATED_AT": data.get("last_themes_updated_at", ""),
                "Chemin": data.get("path_fr", "")
            }

            period_file = str(data.get("time_period", ""))+ ".json"
            annee = str(data.get("time_period_start", ""))[:4]
            chemin_period = os.path.join(chemin, annee)
            os.makedirs(chemin_period, exist_ok=True)

            # Sauvegarde JSON
            fichier_json = os.path.join(
                chemin_period,
                period_file.replace(' ', '_')
            )

            with open(fichier_json, "w", encoding="utf-8") as f:
                json.dump(data_json, f, ensure_ascii=False, indent=4)

            print(f"Données sauvegardées : {fichier_json}")
```

Figure 5: Dernière partie : Fin du traitement et rangement dans les dossiers en fonction de l'année

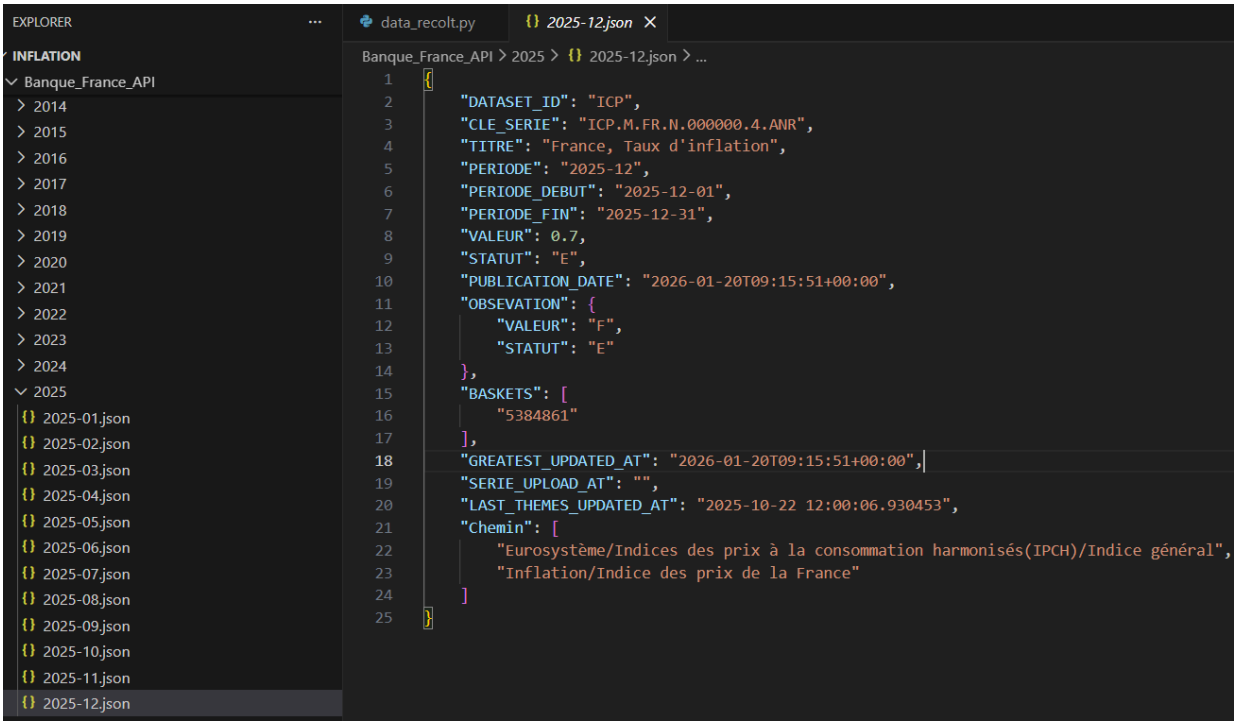


Figure 6: L'arborescence des fichiers avec dedans l'un des fichiers JSON retournés et au bon format

Après avoir réussi à obtenir les données sous le bon format et avec aucune erreur ou problème au niveau de la récupération, on adapte le code originaire du local vers GCP, en ajoutant les dossiers du Bucket et la bibliothèque Python google.cloud

Google Cloud Platform (GCP) est la plateforme de services cloud proposée par Google. Elle fournit des ressources informatiques accessibles à la demande : calcul, stockage, bases de données, IA, sécurité, outils DevOps, etc.

Plus simplement, GCP permet aux entreprises de déployer des applications, stocker des données, analyser de grands volumes d'information, ou encore faire tourner des modèles d'IA via Vertex AI (Vertex AI, s.d.), sans gérer elles-mêmes des serveurs physiques.

GCP repose sur des datacenters hyperscale, ce sont en quelques sortes des usines géantes de données qui sont conçues pour des hébergements à une échelle massive et quasiment infinie, ces derniers sont capables de fournir des ressources massives et élastiques. Les hyperscalers sont définis comme des fournisseurs capables d'offrir du cloud computing et du stockage à très grande échelle, comme Google Cloud, AWS, Azure, IBM Cloud ou Alibaba Cloud

Ses principaux concurrents sont : AWS (Amazon Web Services) et Microsoft Azure

Logical Conseils choisit GCP parce que les projets data internes sont hébergés localement sur GCP.

```
import os
import requests
import pprint
import time
import json
from google.cloud import storage

# Configuration
BUCKET_NAME="inflation_bucket_1"
FOLDER="inflation"

# Initialisation client storage
CLIENT=storage.Client()

BUCKET=CLIENT.get_bucket(BUCKET_NAME)
```

Figure 7: Première partie où on configure le bucket et on importe la bibliothèque Google Cloud

```
chemin=f"{FOLDER}/{annee}/{period_file}.json"
blob=BUCKET.blob(chemin)
blob.upload_from_string(data=json.dumps(data_json, ensure_ascii=False), content_type="application/json")
```

Figure 8 : Seconde partie où on modifiera le chemin afin de mettre le résultat dans le bucket

Après on pourra mettre le code sur Github, où il y a un dépôt Git contenant le travail actuel

GitHub est une plateforme en ligne de gestion de code source basée sur Git, un système de contrôle de version. Elle permet :

- D'héberger des projets (code, documents, fichiers techniques)
- De collaborer à plusieurs sur un même projet
- De suivre l'historique des modifications
- De gérer des versions, des branches et des contributions
- De partager du code publiquement ou en privé

GitHub est aujourd'hui l'un des outils les plus utilisés par les développeurs, les entreprises et les équipes DevOps.

Un dépôt (repository) GitHub est organisé comme un **arbre de fichiers**, similaire à ce que l'on trouve sur un ordinateur.

Voici un schéma d'une structure typique :

nom-du-projet/

```
├── README.md
├── LICENSE
├── .gitignore
│
├── src/
│   ├── main.py
│   ├── utils.py
│   └── ...
│
├── docs/
│   ├── manuel.pdf
│   └── schema.png
│
├── tests/
│   ├── test_main.py
│   └── ...
│
└── data/
    ├── input/
    └── output/
```

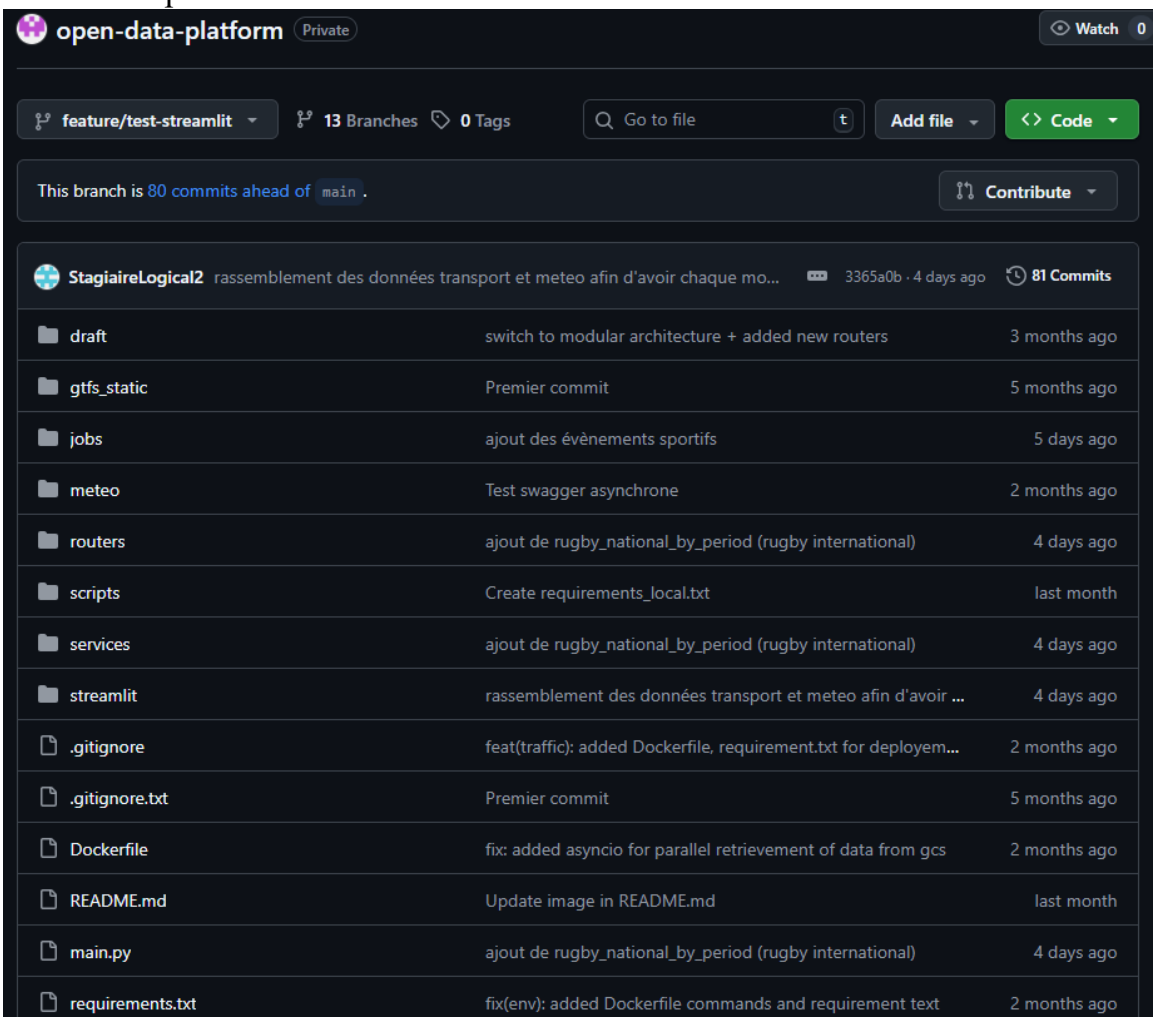


Figure 9 : Répertoire de la branche feature/test-streamlit et son arborescence de fichiers

Pour chaque donnée nouvelle que l'on souhaite étudier et tester on doit créer une nouvelle branche basée sur la branche dev qui correspond à la branche de production de l'application.

Figure 10: Les différentes branches

Sur Google Cloud Platform, il est possible de lancer l'éditeur Visual Studio Code directement depuis la console via le Cloud Shell. Une fois sur l'interface, on peut récupérer notre projet GitHub et gérer son cycle de développement (changement de branche avec git checkout ou création avec git branch) afin de conserver ses modifications sur le stockage persistant du Cloud Shell.

Maintenant que le dépôt Git est initialisé sur Cloud Run, on peut aborder la partie structure du projet, permettant de savoir quelle fonctionnalité est utilisée pour quelle partie du projet.
On peut distinguer 4 grandes parties dans le Projet OpenData:

- Les jobs : Partie Logique Métier permettant de créer les codes récoltant les données des APIs et que l'on va définir une période où le job s'activera et récoltera les données via un Cron, en fonction des données cela peut être en jours ou minutes, ou bien même en mois.
- Les routers : Partie API qui permet de faire des routes et de faire fonctionner l'API Open Data
- Les services : Partie permettant de coder les fonctions liées aux routers et à Streamlit
- Streamlit : Partie Applicative permettant de visualiser le projet

Les technologies utilisées jusqu'à présent furent uniquement Python et quelques bibliothèques comme requests, google.cloud ou encore Json. Mais pour utiliser et interagir avec les APIs, on devra utiliser Fast Api et par la suite pour l'application graphique, Streamlit.

Fast API est un Framework web Python, qui est utilisé pour créer rapidement et simplement des Apis, permettant également de les mettre en production. Il s'appuie sur les annotations de types de Python afin d'avoir :

- Une validation automatique des données
- Une exécution asynchrone
- Une génération automatique de documentation interactive
- Une rapidité de développement grâce à la détection d'erreurs et à de l'autocomplétion

Le lien de la documentation se trouve sur ce lien : **https://fastapi.tiangolo.com**

Streamlit est un framework open-source écrit en Python, conçu pour la création rapide d'applications web interactives dédiées à la Data Science et au Machine Learning. Contrairement aux frameworks web traditionnels (comme Django ou Flask) qui séparent strictement le "Back-end" du "Front-end", Streamlit adopte une approche de développement déclaratif. Il permet aux ingénieurs de transformer des scripts de traitement de données en interfaces utilisateur (UI) sophistiquées en utilisant exclusivement le langage Python, sans nécessiter de compétences en HTML, CSS ou JavaScript.

Le code de streamlit est globalement du Python, mais avec des ajouts qui lui sont propres afin d'avoir une interface propre et simple :

Pour les Textes

- `st.text()` : Texte
- `st.title()` : Titre de l'application
- `st.header()`: En-tête d'une section
- `st.markdown()`: Démarquage d'une section
- `st.subheader()`: Sous-titre d'une section
- `st.caption()`: Légende
- `st.code()`: Code
- `st.latex()`: Expressions mathématiques formatées en LaTeX. ex: st.latex(r'" a + b = c ")

Pour les Medias

- `st.image()`: Image
- `st.audio()`: Audio
- `st.video()`: Vidéo

Pour les Saisies

- `st.number_input()`: Saisie numérique
- `st.text_input()`: Saisie textuelle
- `st.date_input()`: Saisie de date
- `st.time_input()`: Saisie horaire
- `st.text_area()`: Saisie de texte avec moyen d'écrire
- `st.file_uploader()`: Téléchargement de fichiers

```
open-data/
├── draft/
├── gtfs_static/
├── jobs/
│   ├── cinema/
│   ├── commodity/
│   ├── covid/
│   ├── inflation/
│   ├── sports/
│   ├── traffic/
│   └── transport/
├── meteo/
├── routers/
├── scripts/
├── services/
│   ├── __init__.py
│   ├── gcs.py
│   └── logger.py
├── streamlit/
│   ├── Dockerfile
│   ├── main.py
│   ├── requirements.txt
│   └── style.css
├── Dockerfile
├── main.py
├── README.md
└── requirements.txt
```

Figure 11: Structure du Git et du projet Open Data

On mettra pour chaque type de donnée exploitable, un dossier correspondant dans le dossier `jobs`, dans ce dossier on mettra le code qui récupère les données depuis l'API dans un fichier qui s'appellera `main.py`, il sera accompagné d'un fichier `requirements.txt` qui sera présent pour pouvoir y marquer les dépendances Python comme les bibliothèques et avec un fichier `Dockerfile` qui sera là pour pouvoir exécuter le code tout en installant les dépendances issues de l'environnement virtuel.

Un environnement virtuel Python est un outil d'isolation permettant de créer des espaces d'exécution hermétiques pour chaque projet. Contrairement à une installation globale, l'environnement virtuel encapsule ses propres binaires et sa propre arborescence de bibliothèques. Cela nous permet de ne pas avoir de problèmes liés aux installations des bibliothèques.

Sur GCP, afin de faire tourner le code efficacement on a des fonctions plutôt utiles afin de réaliser cela :
- GCP possède une architecture non structurée à la manière d'un Data Lake (Microsoft Azure, s.d.) au contrairement d'un Data WareHouse comme SnowFlake possède une architecture structurée (Oracle France, s.d.) .
- On a la possibilité de créer des images Docker et de les stocker via Artefact Registry, c'est ce qu'on utilisera afin de créer des jobs et des services.
- Les jobs sont créer pour stocker la logique métier soit par exemple les codes utilisés

pour récupérer les données.

- Les services sont créés afin de pouvoir visualiser le travail fourni et de le vérifier via des logs.

Afin de créer un job, on créera une image Docker sur Artifact Registry.

Explication de ce qu'est Artifact Registry :

Artifact Registry est un service de Google Cloud qui sert à stocker et gérer des artefacts logiciels, c'est-à-dire tout ce dont on a besoin pour déployer une application comme des images Docker ou des paquets comme Python, Maven pour Java ou npm pour React.

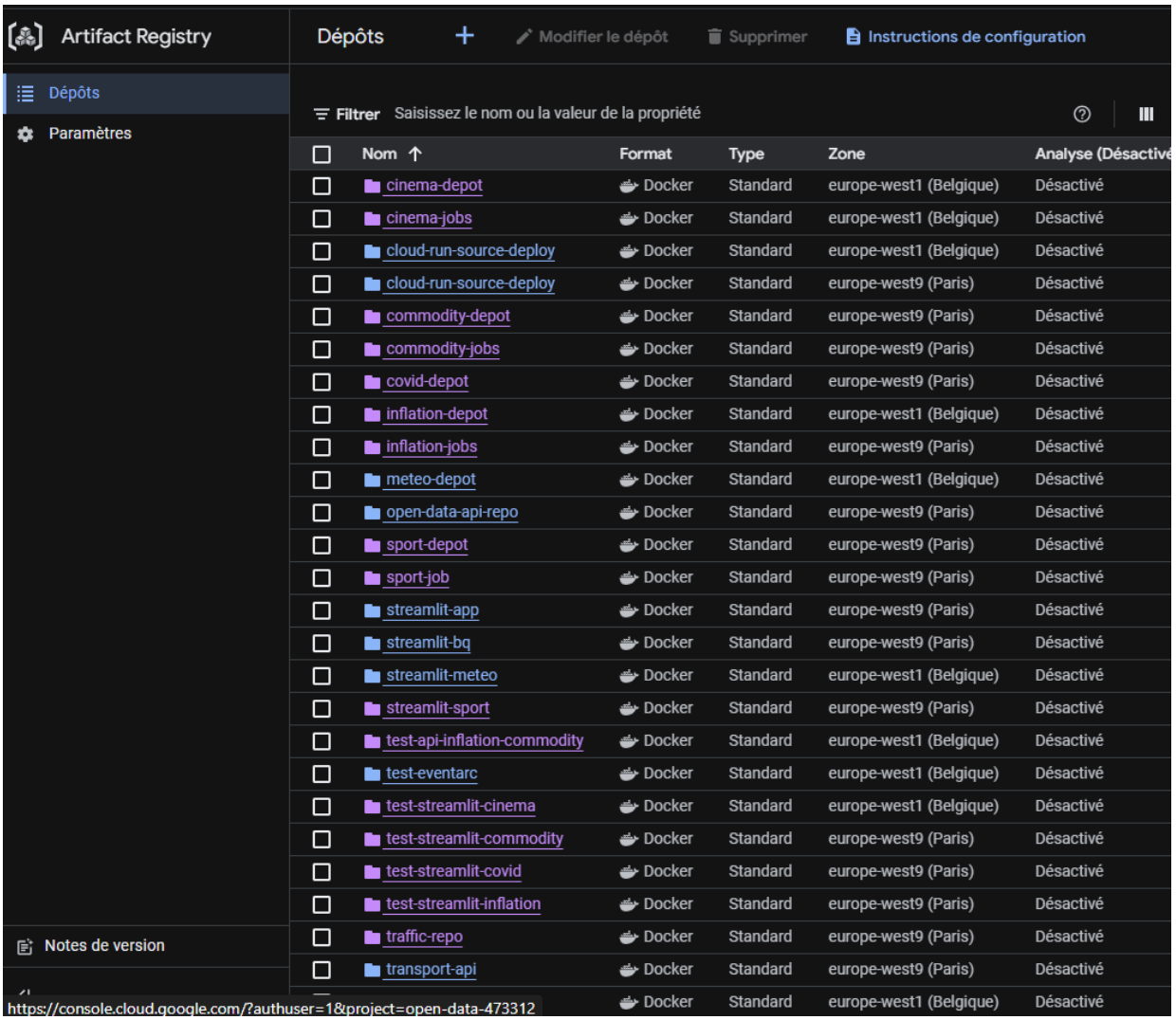


Figure 12: Interface d'Artifact Registry, ici on peut y voir les dépôts, leur zone et également le moyen d'en créer via la croix +

Artifact Registry / Créer un dépôt

Dépôts
Paramètres

Créer un dépôt

Nom *

Format
Docker

Mode
Standard
Distant
Virtuel

Type d'emplacement
Région
Multirégional

Région *

Description

Étiquettes
+ Ajouter une étiquette

Chiffrement
Par défaut, cette ressource est chiffrée à l'aide d'une clé gérée par Google. Si vous devez gérer votre chiffrement, vous pouvez utiliser une clé gérée par le client à la place. En savoir plus

Clé de chiffrement gérée par Google
Clés appartenant à Google

Clé Cloud KMS
Clés appartenant aux clients

Notes de version

Vous pouvez désormais automatiser la création des clés Cloud KMS à l'aide d'Autokey.

Figure 13: Ici on peut voir la manière dont on créer le dépôt, en y déposant un nom et en décidant de la zone du serveur, plus le serveur est proche de nous géographiquement, moins les coûts seront onéreux.

Après avoir créé notre dépôt, on ira créer un Bucket

Cloud Storage

Présentation
Buckets
Surveillance
Paramètres
Storage Intelligence
Ensembles de données Insigh...
Configuration

Buckets Créer Actualiser Accéder au chemin Apprendre

Filtrer Filtrer les buckets Afficher les options

Nom	Création	Type d'emplacement	Zone	Classe de stockage par défaut	Dernière mod
cinema_bucket	29 janv. 2026, 09:10:00	Multi-region	us	Standard	29 janv. 2026,
commodity_bucket	8 janv. 2026, 15:26:31	Region	europe-west9	Standard	8 janv. 2026,
covid_bucket_france	2 févr. 2026, 14:50:08	Multi-region	us	Standard	2 févr. 2026, 1
gcf-v2-sources-178258885645-europ...	5 févr. 2026, 10:49:25	Region	europe-west9	Standard	5 févr. 2026, 1
gcf-v2-uploads-178258885645.europ...	5 févr. 2026, 10:49:19	Region	europe-west9	Standard	5 févr. 2026, 1
inflation_bucket_1	22 janv. 2026, 10:00:24	Region	europe-west9	Standard	22 janv. 2026,
meteo_opendata	29 sept. 2025, 16:29:09	Multi-region	eu	Standard	29 sept. 2025
open-data-473312_cloudbuild	28 oct. 2025, 11:30:10	Multi-region	us	Standard	28 oct. 2025,
run-sources-open-data-473312-europ...	28 oct. 2025, 10:45:23	Region	europe-west1	Standard	28 oct. 2025,
run-sources-open-data-473312-europ...	26 sept. 2025, 15:18:25	Region	europe-west9	Standard	26 sept. 2025
sport_bucket_france	9 févr. 2026, 10:59:20	Region	europe-west9	Standard	9 févr. 2026, 1
traffic_bucket_1	10 déc. 2025, 14:37:42	Region	europe-west9	Standard	10 déc. 2025,
transport_bucket_1	26 sept. 2025, 15:07:31	Region	europe-west9	Standard	6 févr. 2026,

Lignes par page: 30 1 – 13 sur 13

Figure 14: Voici la liste des buckets

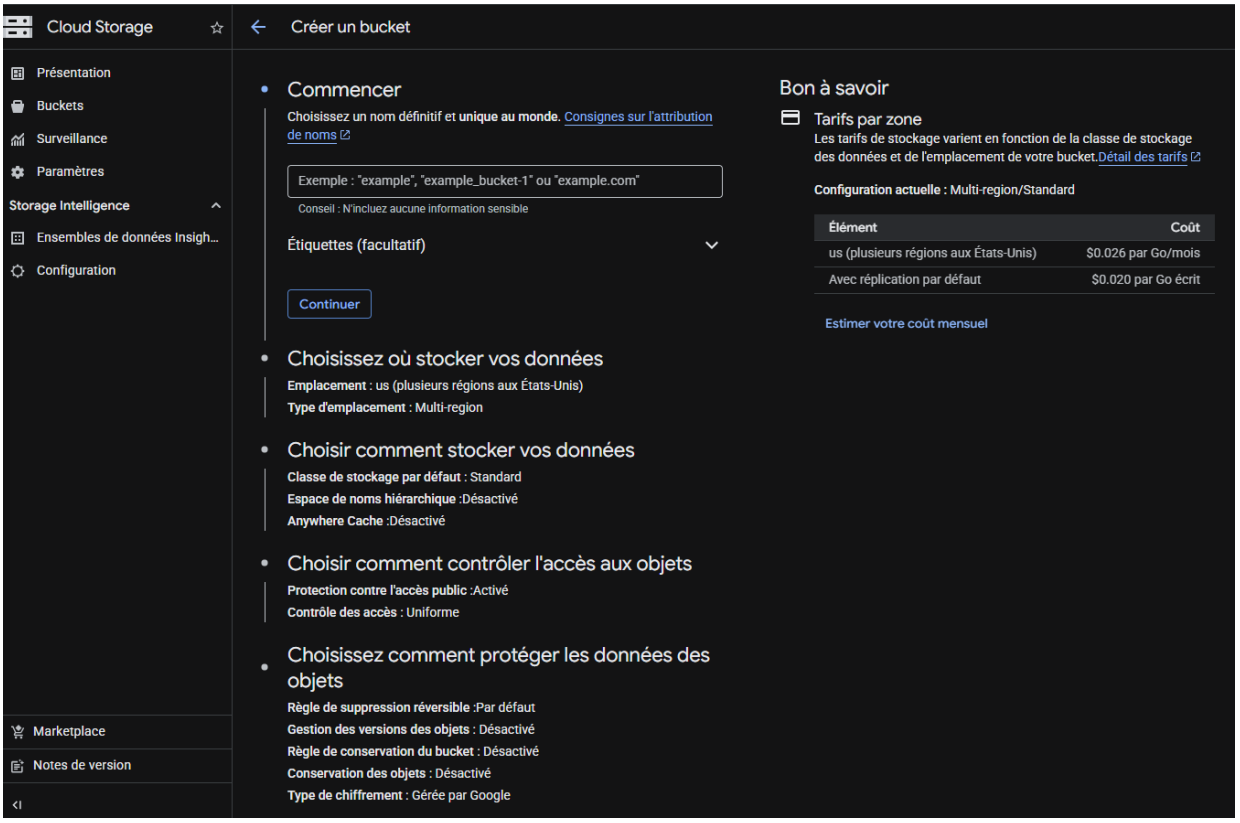


Figure 15: Interface de création d'un Bucket

Après l'avoir créé en mettant le nom et la zone où sera stocké le serveur, on retourne sur notre dossier jobs puis dans le dossier voulu en ligne de commande afin de pouvoir ajouter le code dans un Dockerfile, qui sera utilisé afin de faire tourner le code avec cette commande :

```
gcloud builds submit --tag REGION-
docker.pkg.dev/PROJECT_ID/REPO_NAME/IMAGE_NAME:v1
```

Dès que le code sera mis dans le job, ce dernier sera exécuté un nombre de fois défini via une expression cron. Une expression CRON est une chaîne de caractères généralement composée de cinq ou six champs, qui définit un calendrier d'exécution pour des tâches automatisées (Voir Annexe sur les Crons). Par la suite, dans ce job, on créera un swagger permettant d'interroger l'API afin d'obtenir les données voulues, les données voulues seront prises en fonction du moment, comme une date, une période ou même en temps réel. Pour vérifier le résultat du swagger, on déploiera un premier service qui sera d'abord un swagger

Afin de le créer on doit s'attarder sur la partie routers de notre projet :

Les routers sont simplement les routes qui vont définir les endpoints de notre API. Chaque endpoint peut être appelé avec des paramètres obligatoires représentés par des * ou bien des paramètres facultatifs :

- By Period : Mesure dans laquelle l'utilisateur souhaite un ordonnancement par période en prenant au minimum 2 dates et au maximum 2 dates et un ou plusieurs éléments supplémentaires comme un type ou un département.

- By Date : Mesure dans laquelle l'utilisateur souhaite un ordonnancement par dateen prenant au minimum 1 date et au maximum 1 date et un ou plusieurs éléments supplémentaires comme un type ou un département.

- By Now : Mesure où l'utilisateur souhaite les données en temps réel, il pourra soit directement avoir accès aux données soit, il faudra saisir une variable supplémentaire comme un département ou un type

- By Type : Mesure où l'utilisateur souhaite les données en fonction du type des données.

open-data > open-data-platform > routers > commodity_by_type_latest.py > {} APIRouter

```python
from fastapi import APIRouter, HTTPException
from fastapi.responses import StreamingResponse
from services.gcs import get_latest_blob_for_commodity_type

router = APIRouter(prefix="/commodity", tags=["commodity"])

@router.get("/{commodity_type}/latest/download")
def download_latest(commodity_type: str):
    blob = get_latest_blob_for_commodity_type(commodity_type)

    if blob is None:
        raise HTTPException(status_code=404, detail="No JSON found")

    stream = blob.open("rb")

    return StreamingResponse(
        stream,
        media_type="application/json",
        headers={"Content-Disposition": f"attachment; filename={blob.name.split('/')[-1]}"}
    )
```

Figure 16: Exemple de code de routers : commodity_by_type_latest.py pour les matières premières en fonction de leur type

Dans le dossier services, il y a un dossier appelé gcs.py faisant office de fichier de stockage de toutes les fonctions liées aux routers afin que lors des appels des APIs, les données sont bien envoyées et soient opérationnelles.

```python
def get_latest_blob_for_commodity_type(commodity_type: str):
    prefix = f"{FOLDER_COMMODITY}/{commodity_type}.json"

    blobs = client.list_blobs(
        BUCKET_NAME_COMMODITY,
        prefix=prefix
    )

    latest_blob = None

    for blob in blobs:
        if blob.name.endswith(".json"):
            latest_blob = blob
            break  # on s'arrête dès qu'on trouve le fichier

    if latest_blob:
        logger.info(f"File found for commodity '{commodity_type}': {latest_blob.name}")
    else:
        logger.info(f"No JSON file found for commodity '{commodity_type}'")

    return latest_blob
```

Figure 17: Code lié à commodity_by_type_latest.py afin de pouvoir télécharger sous un unique fichier Json, la concaténation de plusieurs Json, lors de l'appel

Après avoir complété sa fonction, on termine la partie de configuration du swagger en allant sur le fichier main.py qui est à la racine du projet pour importer chaque route du swagger.

open-data > open-data-platform > main.py > ...

```python
from fastapi import FastAPI
from routers.transport_now import router as now_router
from routers.transport_by_date import router as date_router
from routers.transport_by_period import router as period_router
from routers.commodity_by_type_latest import router as commodity_by_type_latest
import time
from services.logger import logger
from routers.traffic_now import router as traffic_now
from routers.inflation_date import router as inflation_date
from routers.inflation_period import router as inflation_period
from routers.cinema_by_date import router as cinema_by_date
from routers.cinema_by_period import router as cinema_by_period
from routers.cinema_now import router as cinema_now
import os
from routers.foot_now import router as foot_now
from routers.rugby_regional import router as rugby_regional
from routers.rugby_national import router as rugby_national
from routers.rugby_national_now import router as rugby_national_now
from routers.rugby_regional_by_period import router as rugby_regional_by_period
from routers.rugby_national_by_period import router as rugby_national_by_period


start = time.time()

app = FastAPI()

app.include_router(now_router)
app.include_router(date_router)
app.include_router(period_router)
app.include_router(traffic_now)
app.include_router(commodity_by_type_latest)
app.include_router(inflation_date)
app.include_router(inflation_period)
app.include_router(cinema_by_date)
app.include_router(cinema_by_period)
app.include_router(cinema_now)
app.include_router(foot_now)
app.include_router(rugby_regional)
app.include_router(rugby_regional_by_period)
app.include_router(rugby_national)
app.include_router(rugby_national_now)
app.include_router(rugby_national_by_period)
```

Figure 18 : Ici on importe les différentes routes

open-data > open-data-platform > main.py > ...

```python
start = time.time()

app = FastAPI()

app.include_router(now_router)
app.include_router(date_router)
app.include_router(period_router)
app.include_router(traffic_now)
app.include_router(commodity_by_type_latest)
app.include_router(inflation_date)
app.include_router(inflation_period)
app.include_router(cinema_by_date)
app.include_router(cinema_by_period)
app.include_router(cinema_now)
app.include_router(foot_now)
app.include_router(rugby_regional)
app.include_router(rugby_regional_by_period)
app.include_router(rugby_national)
app.include_router(rugby_national_now)
app.include_router(rugby_national_by_period)



@app.on_event("startup")
async def startup_event():
    logger.info("Starting Transport API...")
    logger.info(f"Startup time: {time.time() - start:.4f}s")

@app.on_event("shutdown")
async def shutdown_event():
    logger.info("Shutting down Transport API...")

@app.get("/")
def root():
    return {"message": "Welcome to the Sport API!"}

if __name__ == "__main__":
    import uvicorn
    port = int(os.environ.get("PORT", 8080))
    uvicorn.run("main:app", host="0.0.0.0", port=port)
```

Figure 19: Ici on initialise le swagger

Après un déploiement via la commande :
`gcloud builds submit --tag REGION-docker.pkg.dev/PROJECT_ID/REPO_NAME/IMAGE_NAME:v1`

Puis avec l'interface GCP, où il faudra mettre à jour le Dockerfile

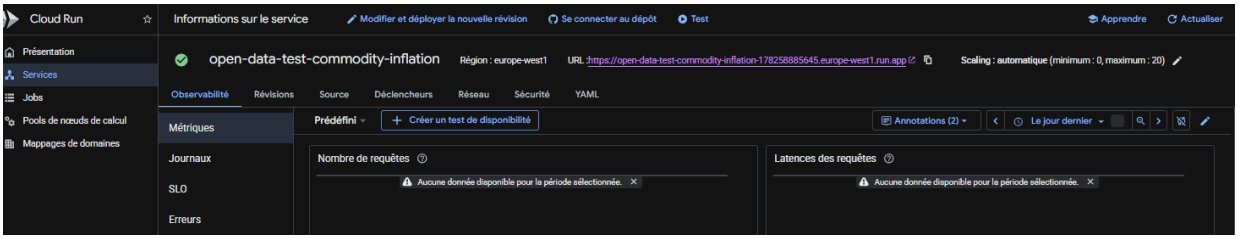


Figure 20: Interface GCP, où on a les informations du service

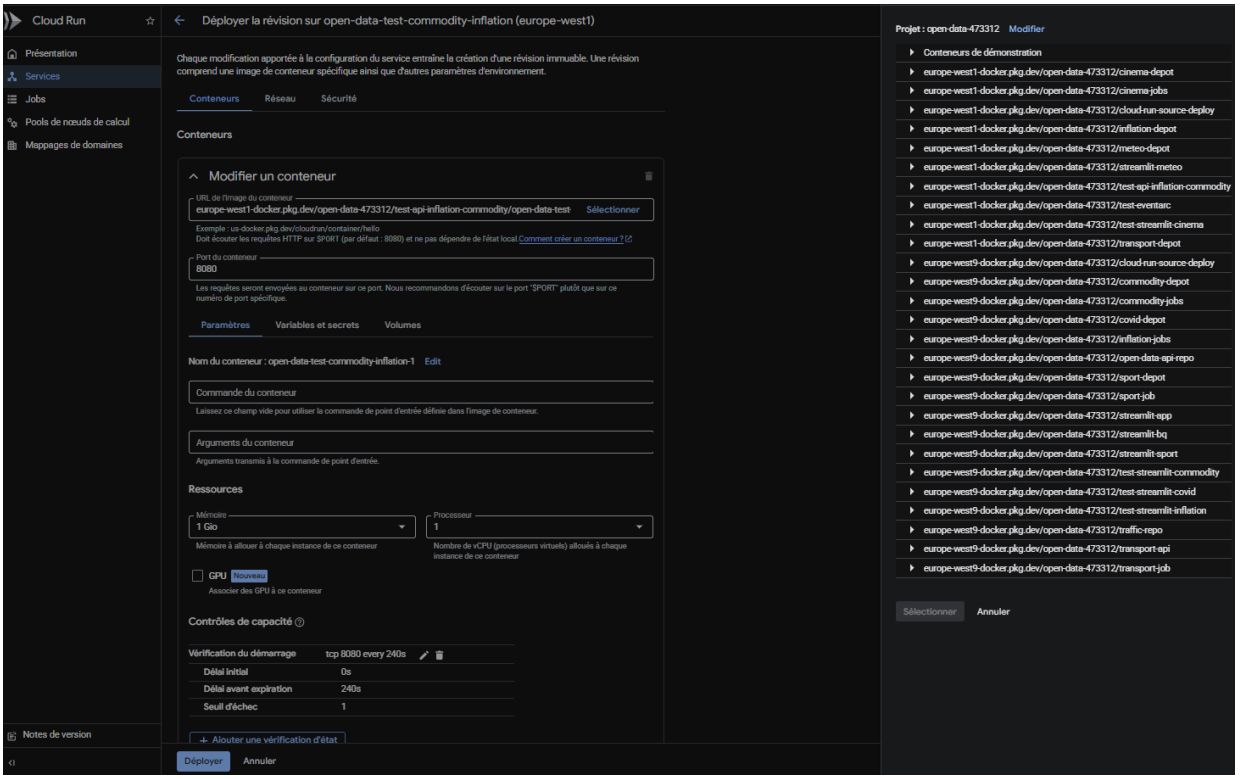


Figure 21: Interface GCP, où il faudra mettre à jour le Dockerfile du service

On a désormais accès à l'interface du Swagger

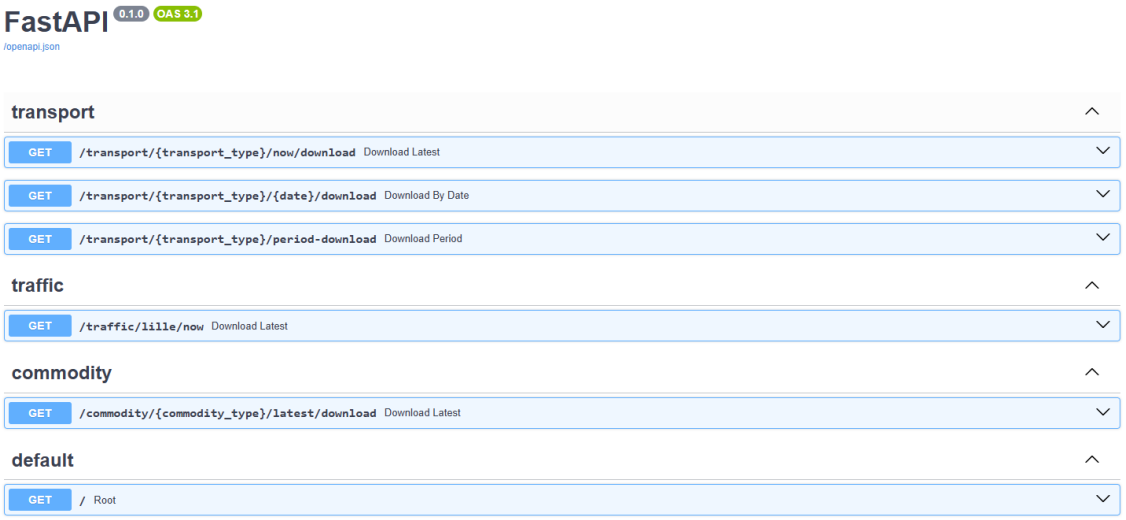


Figure 22: Interface du Swagger

On peut ouvrir l'un des onglets et interroger l'API afin d'avoir les données

GET /commodity/{commodity_type}/latest/download Download Latest

Parameters Cancel

Name Description

commodity_type * required
string
(path)
argent

Execute

Responses

Code Description Links

200 Successful Response No links

Media type
application/json
Controls Accept header.

Example Value | Schema

```
"string"
```

422 Validation Error No links

Media type
application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
```

Figure 23: Test de l'API Commodity (Matières Premières) afin d'avoir les données sur le cours de l'argent

GET /commodity/{commodity_type}/latest/download Download Latest

Parameters Cancel

Name Description

commodity_type * required
string
(path)
argent

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://open-data-api-178258885645.europe-west1.run.app/commodity/argent/latest/download' \
  -H 'accept: application/json'
```

Request URL

```
https://open-data-api-178258885645.europe-west1.run.app/commodity/argent/latest/download
```

Server response

Code Details

200 Response body
Download file

Response headers

```
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
content-disposition: attachment; filename=argent.json
content-type: application/json
date: Mon,16 Feb 2026 13:27:16 GMT
server: Google Frontend
```

Responses

Figure 24: La réponse de l'API lorsque c'est un succès nous renvoie un fichier téléchargeable Download file

Autrement une erreur 500 apparaîtra dans le cas où le code possède une erreur et une erreur 404 apparaîtra pour dire qu'il n'y a pas de données récupérables à cet instant

(Voir Annexe sur les Erreurs pour la spécification des erreurs possibles)

En cliquant sur le lien, on télécharge le dossier en JSON et on obtient le résultat suivant


```
C: > Users > s.quesnoy > Downloads > {} argent (3).json > ...
1  {
2    "metadata": {
3      "LAST_UPDATE": "2026-01-27",
4      "REF_AREA": "ETR"
5    },
6    "data": [
7      {
8        "Date": "2025-12",
9        "Valeur": 361.4,
10       "Statut": "A",
11       "Qualite": "DEF"
12     },
13     {
14       "Date": "2025-11",
15       "Valeur": 288.7,
16       "Statut": "A",
17       "Qualite": "DEF"
18     },
19     {
20       "Date": "2025-10",
21       "Valeur": 279.7,
22       "Statut": "A",
23       "Qualite": "DEF"
24     },
25     {
26       "Date": "2025-09",
27       "Valeur": 238.6,
28       "Statut": "A",
29       "Qualite": "DEF"
30     },
31     {
32       "Date": "2025-08",
33       "Valeur": 214.9,
34       "Statut": "A",
35       "Qualite": "DEF"
36     },
37     {
```


Figure 25: Résultat sur la valeur de l'argent en fonction d'une période d'un mois

Après avoir vérifié que le résultat donné par le swagger est correcte, on peut passer à la partie applicatif et graphique du projet : Streamlit

A la manière des autres parties, le dossier Streamlit du projet, possède un fichier `main.py`, un `requirements.txt` et un `Dockerfile`, mais en plus il peut avoir un fichier css qui permet de rajouter un style supplémentaire à la page

Mais bien que le CSS n'est pas l'élément le plus important dans le projet, j'ai eu la tâche d'en créer un avec les couleurs de l'entreprise.

Afin de l'importer efficacement j'ai dû utiliser ces deux lignes :

```
with open("style.css") as f:
    st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True)
```

Figure 26: Code pour importer le css

La code de la feuille de style n'est pas si différente que celle que l'on peut trouver en développement web classique, à la différence qu'au lieu de mettre les balises ou d'autres éléments issus du code, il faut plutôt définir les éléments voulus de la manière suivante :

```
[data-testid= "element"]
```

Par exemple pour appliquer un style à la barre latérale, il faudra utiliser :

```
[data-testid= "stSidebar"]
```

Avec un élément complètement décoré on obtient ceci :

```
/* Sidebar décorative */
[data-testid="stSidebar"] {
    background: linear-gradient(
        180deg,
        var(--c4) 0%,
        var(--c5) 30%,
        var(--c6) 60%,
        var(--c7) 90%
    );
    color: var(--c1);
    padding: 20px;
    border-radius: 0;
}
```

Figure 27: Code CSS pour une Sidebar

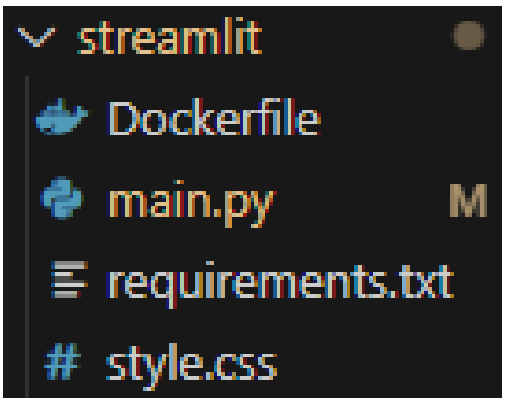


Figure 28: Dossier de la partie Streamlit

Etant donné que l'application en production n'est pas modifiée afin de ne rien altérer, les captures d'écran suivantes proviennent d'une version créée afin de se rapprocher le plus possible du résultat lors de la mise en production

```
open-data > open-data-platform > streamlit > main.py > {} json
1   # streamlit_app.py
2
3   import streamlit as st
4   import requests
5   import json
6   import pandas as pd
7   import json
8   import datetime
9   import numpy as np
10
11  # ==============================================================================
12  # Configuration
13  # ==============================================================================
14
15  API_TRANSPORT = "https://open-data-api-178258885645.europe-west1.run.app"
16  API_METEO = "https://open-data-api-178258885645.europe-west9.run.app"
17  API_COMMODITY= "https://open-data-api-178258885645.europe-west1.run.app"
18  API_INFLATION = "https://inflation-api-178258885645.europe-west1.run.app"
19  API_CINEMAS = "https://cinema-api-178258885645.europe-west1.run.app"
20  API_COVID = "https://covid-depot-178258885645.europe-west9.run.app"
21  API_SPORT = "https://sport-178258885645.europe-west9.run.app"
22  API_EVENTS = "https://events-test-swagger-178258885645.europe-west9.run.app"
23
```

Figure 29: Début du code Streamlit avec l'importation de toutes les APIs

On définit après les variables liées à chaque API

```python
# ----------------------------
# VARIABLES POUR API TRANSPORTS
# ----------------------------
TRANSPORT_TYPES = ["bus", "metro", "tram"]
moments_transport = ["Maintenant", "Par Période", "Par Date"]

# ----------------------------
# VARIABLES POUR API COMMODITY
# ----------------------------
COMMODITY_CATEGORIES = [
    "argent",
    "petrole_brut",
    "aluminium",
    "cuivre",
    "etain",
    "plomb",
    "zinc",
    "or",
    "platine",
    "cobalt"
]

# ----------------------------
# VARIABLES POUR API INFLATION
# ----------------------------
moments_inflation = ["Par Période", "Par Date"]

# ----------------------------
# VARIABLES POUR API COVID
# ----------------------------
moments_covid = ["Par Période", "Par Date"]

# ----------------------------
# VARIABLES POUR API METEO
# ----------------------------
METEO_CATEGORIES = [
    "temperatures",
    "precipitations",
    "humidite"
```

Figure 30: Partie des variables liées à chaque API

Puis on a la partie configuration des pages de l'application

```python
st.title("Open Data API Interface")

# ==========================================================================
# Sidebar - Navigation
# ==========================================================================

st.sidebar.title("Navigation")

# Vérification de la connexion des APIs
is_connected_transport, api_response_transport = check_api_connection(API_TRANSPORT)
is_connected_meteo, api_response_meteo = check_api_connection(API_METEO)
is_connected_commodities, api_response_commodities = check_api_connection(API_COMMODITY)
is_connected_inflation, api_response_inflation = check_api_connection(API_INFLATION)
is_connected_cinemas, api_response_cinemas = check_api_connection(API_CINEMAS)
is_connected_covid, api_response_covid = check_api_connection(API_COVID)
is_connected_sport, api_response_sport = check_api_connection(API_SPORT)

if is_connected_transport and is_connected_meteo and is_connected_inflation and is_connected_commodities and is_connected_cinemas and is_connected_covid and is_connected_sport:
    st.sidebar.success("APIs connectées")
else:
    st.sidebar.error("APIs non disponibles")

st.sidebar.markdown("---")

router_choice = st.sidebar.radio(
    "Choisir un service :",
    ["Accueil", "Transport", "Traffic Now", "Meteo", "Matières Premières", "Inflation", "Covid", "Evènements"]
)

# On nettoie les données si on change de page
clear_data_on_page_change(router_choice)

# ==========================================================================
# Pages
# ==========================================================================
```

Figure 31: Partie configuration des pages de l'application

Entre les deux parties, il y a de nombreuses fonctions régissant, le nommage des noms, la conversion du Json au DataFrame afin d'avoir les données sous forme de tableau, afin que l'affichage soit plus agréable à l'utilisateur.

Après avoir vérifié le statut des APIs, on va afficher les différentes routes avec quelques explications sur la nature des données possibles à récupérer

```
        st.error("L'API Meteo n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_METEO)

    if is_connected_commodities:
        st.success("L'API Commodity est en ligne et opérationnelle.")
        st.json(api_response_commodities)
    else:
        st.error("L'API Commodity n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_COMMODITY)

    if is_connected_inflation:
        st.success("L'API Inflation est en ligne et opérationnelle.")
        st.json(api_response_inflation)
    else:
        st.error("L'API Inflation n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_INFLATION)

    if is_connected_cinemas:
        st.success("L'API Cinemas est en ligne et opérationnelle.")
        st.json(api_response_cinemas)
    else:
        st.error("L'API Cinemas n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_CINEMAS)

    if is_connected_covid:
        st.success("L'API Covid est en ligne et opérationnelle.")
        st.json(api_response_covid)
    else:
        st.error("L'API Covid n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_COVID)

    if is_connected_sport:
        st.success("L'API Sport est en ligne et opérationnelle.")
        st.json(api_response_sport)
    else:
        st.error("L'API Sport n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_SPORT)

    if is_connected_events:
        st.success("L'API Events est en ligne et opérationnelle.")
        st.json(api_response_events)
    else:
        st.error("L'API Events n'est pas accessible. Vérifiez qu'elle est bien lancée sur " + API_EVENTS)
```

Figure 32: Statut des APIs

```python
router_choice = st.sidebar.radio(
    "Choisir un service :",
    ["Accueil", "Transport", "Traffic Now", "Meteo", "Matières Premières", "Inflation", "Covid", "Evènements"]
)

# On nettoie les données si on change de page
clear_data_on_page_change(router_choice)

# ==========================================================================
# Pages
# ==========================================================================

if router_choice == "Accueil":
    st.header("Bienvenue sur l'interface Open Data API")

    st.markdown("""
    Cette interface vous permet d'interagir avec les APIs Open Data pour récupérer
    des données de transport en commun (bus, métro, tram), de météo, du trafic routier,
    le cours des matières premières, les données évènementielles (cinema, sport) et les données liées au Covid 19 .

    ### Services disponibles

    | Service | Description |
    |---------|------------|
    | **Transport**  | Récupère les données de transport  |
    | **Traffic Now** | Récupère les dernières données de trafic pour Lille |
    | **Meteo** | Récupère les données de météo |
    | **Matières Premières** | Récupère les données des matières premières pour un type spécifique |
    | **Inflation** | Récupère les données de l'inflation |
    | **Covid** | Récupère les données liées au Covid 19 |
    | **Evènements** | Récupère les données évènementielles comme les données de cinéma et de sport |

    ### Statut des APIs
    """)
```

Figure 33: Code de la page d'accueil

Ce qui nous donne en version graphique :

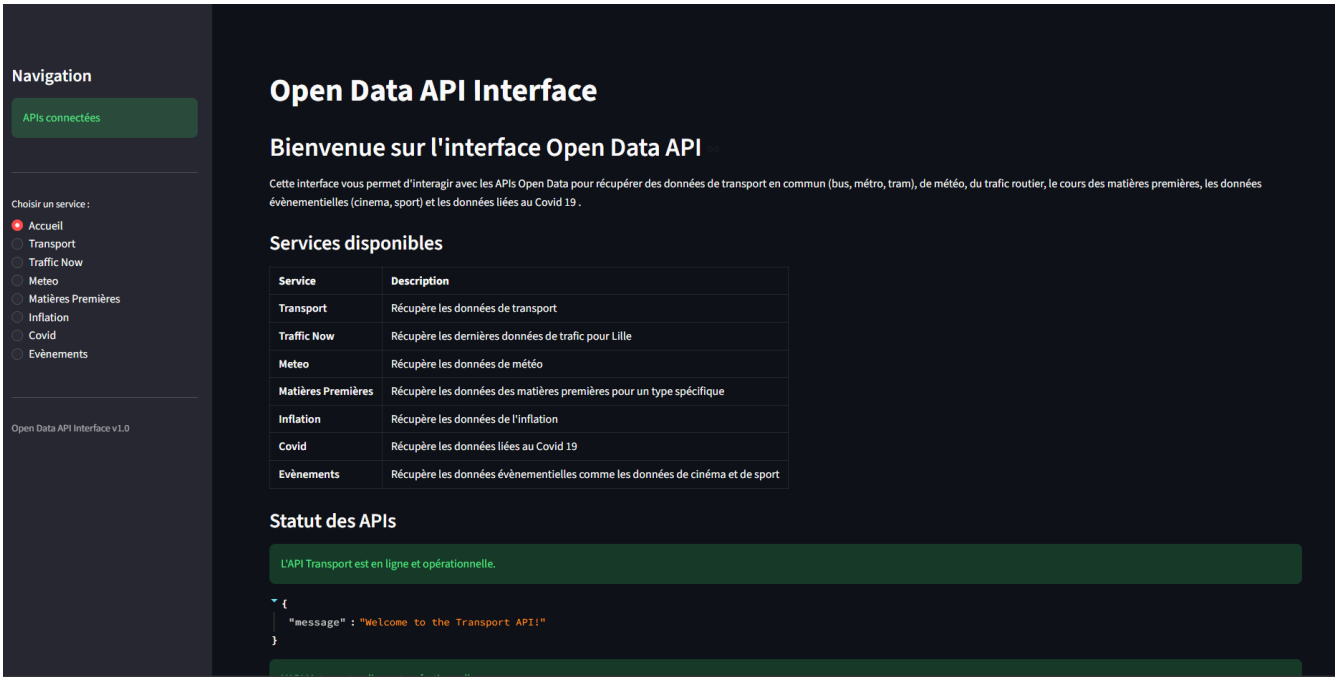


Figure 34: Version Graphique de la page d'accueil

Avec le CSS ajouté, la page devient ceci :

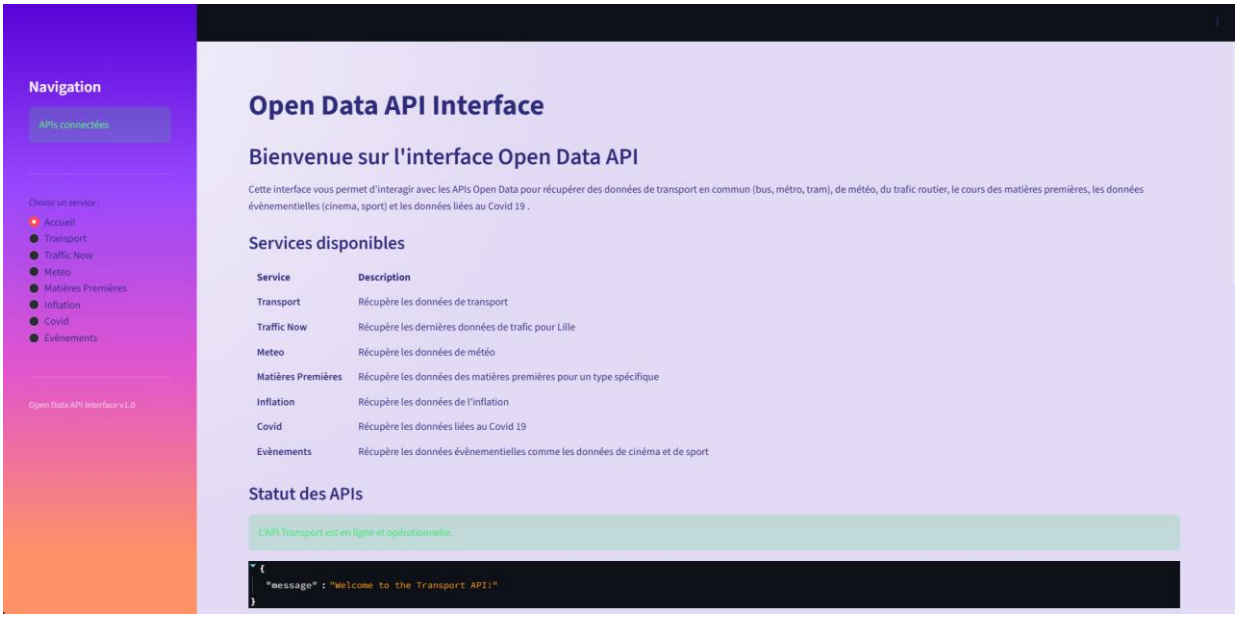


Figure 35: Page d'accueil avec CSS

Pour se diriger vers chaque API c'est via une route que l'on peut y accéder et en fonction de la route, on ira vers le service désiré :

```
elif router_choice == "Evènements":

    evenement = st.selectbox("Type d'Evènement' :", EVENTS)

    if evenement == "Sports":
        st.header("Sports")
        st.markdown("Récupère les données des matchs de différents sports.")

        # Toujours afficher le choix du sport
        sport_type = st.selectbox("Type de Sport :", SPORT_TYPES)

        # Variables par défaut
        endpoint = None
        params = None
        filename = None

        # --------------------------
        # FOOTBALL
        # --------------------------
        if sport_type == "Football":
            sport = "football"

            # Le moment doit TOUJOURS être affiché AVANT toute logique
            moment = st.selectbox("Moment :", moments_football)

            if moment == "Maintenant":
                endpoint = f"/{sport}/latest/download"
                filename = "football_matchs_now.json"
```

Figure 36: Code de la sélection des choix pour l'API des Évènements

Après avoir choisi un service, l'utilisateur aura la possibilité de choisir entre les différentes options qui sont proposées, par exemple pour les Evènements, on en propose de plusieurs sortes :

- Sports
- Cinema
- Evenements Culturels de Métropole Lilloise (Festivals, Atelier, etc ...)

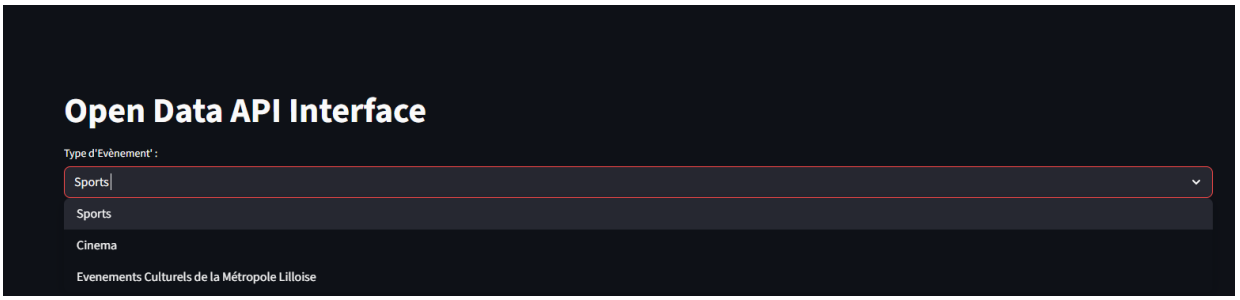


Figure 37: Choix de l'Évènement

En choisissant le Sport, on se retrouve avec d'autres choix de données comme le Football ou le Rugby :

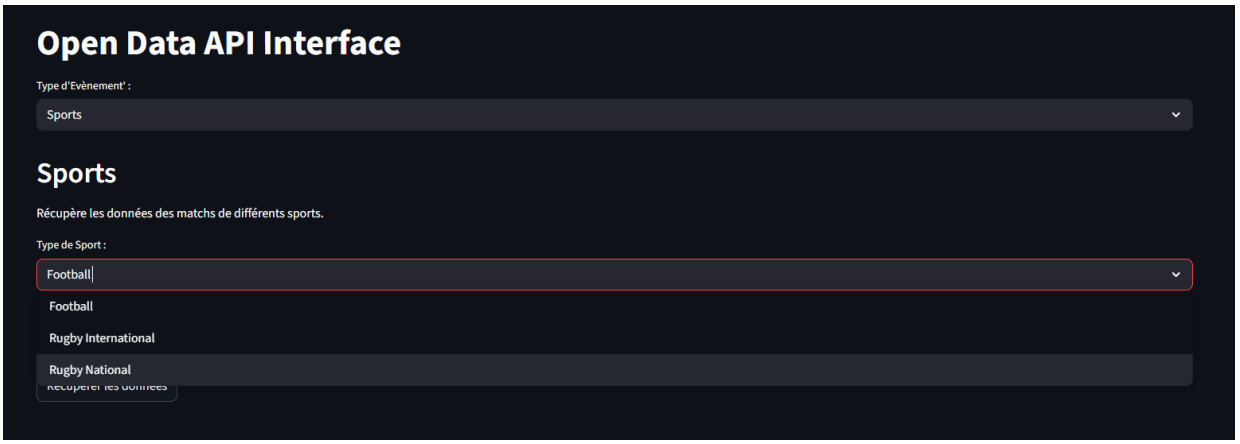


Figure 38: Choix du Sport

En choisissant le Rugby International par exemple, on se retrouve à choisir entre d'autres paramètres afin de savoir la fréquence des données qui doit être renvoyée :

- Maintenant
- Par Période
- Par Date

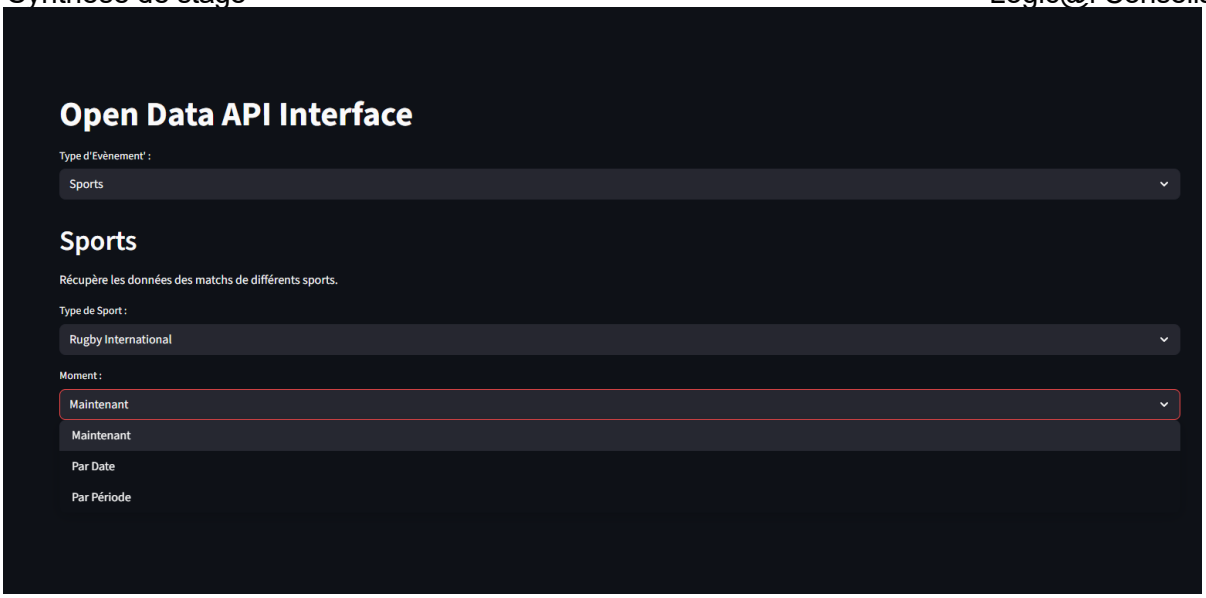
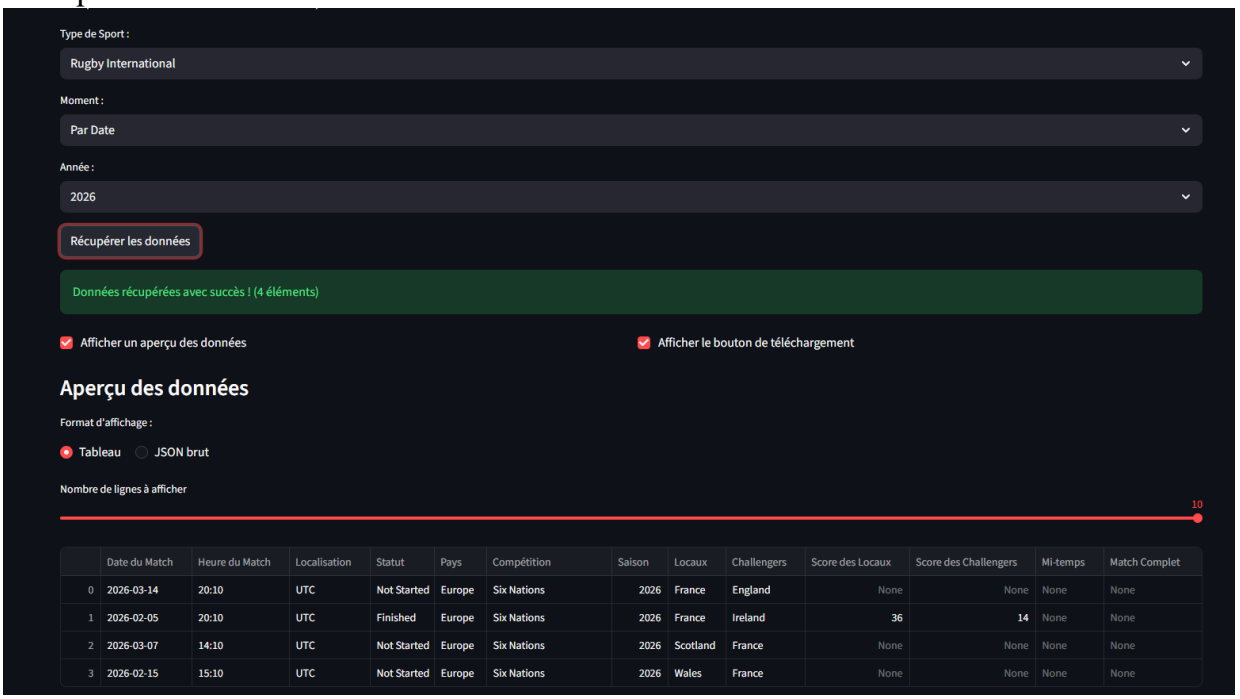


Figure 39: Choix de la période

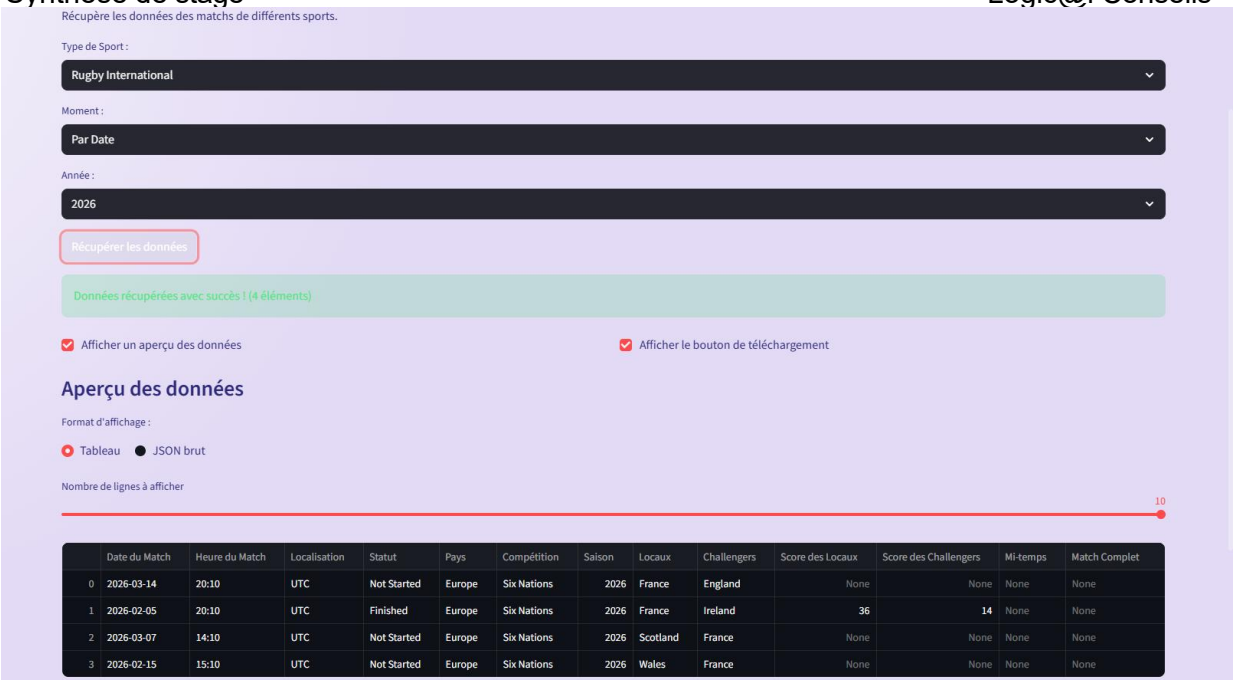
Lorsque l'on effectue une recherche on obtient un tableau de ce format :



	Date du Match	Heure du Match	Localisation	Statut	Pays	Compétition	Saison	Locaux	Challengers	Score des Locaux	Score des Challengers	Mi-temps	Match Complet
0	2026-03-14	20:10	UTC	Not Started	Europe	Six Nations	2026	France	England	None	None	None	None
1	2026-02-05	20:10	UTC	Finished	Europe	Six Nations	2026	France	Ireland	36	14	None	None
2	2026-03-07	14:10	UTC	Not Started	Europe	Six Nations	2026	Scotland	France	None	None	None	None
3	2026-02-15	15:10	UTC	Not Started	Europe	Six Nations	2026	Wales	France	None	None	None	None

Figure 40: Résultat de la recherche

Et avec le CSS, cela devient :



	Date du Match	Heure du Match	Localisation	Statut	Pays	Compétition	Saison	Locaux	Challengers	Score des Locaux	Score des Challengers	Mi-temps	Match Complet
0	2026-03-14	20:10	UTC	Not Started	Europe	Six Nations	2026	France	England	None	None	None	None
1	2026-02-05	20:10	UTC	Finished	Europe	Six Nations	2026	France	Ireland	36	14	None	None
2	2026-03-07	14:10	UTC	Not Started	Europe	Six Nations	2026	Scotland	France	None	None	None	None
3	2026-02-15	15:10	UTC	Not Started	Europe	Six Nations	2026	Wales	France	None	None	None	None

Figure 41: Résultat de la recherche avec le CSS

On peut également le mettre sous format JSON

Figure 42: Vue des données sous le format JSON

Les activités mise en œuvre par rapport au tableau de compétences

Travailler en mode Projet : J'ai pu travailler sur ma réalisation professionnelle en collaboration avec le pôle Recherche et Développement. Grâce à mes collègues j'ai pu avancer et comprendre de mieux en mieux le sujet, également mon maître de stage n'hésitais à faire des points afin de clarifier les consignes et le but dans les tâches que j'avais à faire.

Mettre à disposition des utilisateurs un service informatique : J'ai découvert tout au long du projet Open Data, comment configurer et déployer des services sur GCP.

Organiser son développement personnel : J'ai appris à m'informer sur les nouvelles APIs qui peuvent exister notamment celles du gouvernement ou bien celles libre d'accès. Grâce à mes collègues et mon maître de stage, j'ai pu découvrir et chercher des nouvelles librairies Python, élargissant ma culture sur le langage de programmation.

Les difficultés que j'ai rencontrées et comment j'ai pu trouver des solutions ?

Je rencontrais quelques difficultés à cerner les causes des erreurs issues de Python ou bien de GCP, ou encore parfois je bloquais sur un problème à résoudre comme afficher les données dans une dimension précise, par exemple récolter les données en fonction de date et de bien prendre le contenu de chaque date s'il y a un intervalle compris.

Également j'ai fait face à des limitations vis-à-vis des APIs et de leur recherche, en voulant chercher des APIs convenables et libre d'accès, je me suis confronté à des APIs qui pouvaient être payantes, ou bien encore des APIs qui au bout d'un nombre de requête limité ne renvoyait plus de données. Par exemple l'API SPORT avait un nombre de requête limité et ne donnait pas l'accès à l'ensemble de ses données, mis à part une vingtaine de ligne. J'ai pu aussi constater que certaines APIs n'étaient pas complètes dû à un manque d'entretien et de mise à jour. Par exemple, l'API permettant de récupérer les différents festivals était inexploitable fonctionnellement car les résultats retournés ne permettaient pas de distinguer les évènements qui n'existent plus des évènements qui existent encore

Pour remédier à ces complications, il a fallu que je lise attentivement le code ou les erreurs en question, que je cherche dans les logs de GCP, s'il s'agissait des erreurs liées à GCP lors du déploiement. Je devais également me documenter sur des fonctions en Python comme les expressions régulières ou encore les bibliothèques utilisées afin de s'adapter à la nature des données. Pour les APIs, la documentation était primordiale afin de bien comprendre leur rôle et comment les exploiter.

Les connaissances que j'ai tirées de ce stage

Bien que je connaisse les langages Python et l'utilisation de GitHub et de Postman j'ai su les approfondir grâce à ce stage, via l'utilisation de nombreuses bibliothèques Python comme requests ou numpy.

Avec Github, même si je connaissais déjà, j'ai su en faire une meilleure utilisation qui est plus propre et professionnelle au niveau de la sécurité.

J'ai surtout découvert les Framework Fast API et Streamlit durant le projet Open Data. J'ai aussi découvert par l'intermédiaire de mon maître de stage et de mes collègues du Pôle Recherche et Développement, les nombreuses utilisations de Python, dans de nombreux domaines comme la Data, le Jeu Vidéo ou encore pour des apports dans des applications plus sélectifs comme la géographie via la génération de carte interactive ou encore pour la création de Chatbot via des interfaces simples et efficaces comme Streamlit

J'ai également approfondi mes connaissances sur Google Cloud Plateform (GCP), tout en ayant une meilleure maîtrise et une meilleure compréhension de cette technologie.

J'ai approfondi également ma manière de travailler en équipe et comment bien se coordonner sur les tâches à se répartir.

ANNEXE SUR LES CRONS

Structure d'une expression CRON :

L'expression se lit de gauche à droite, chaque champ étant séparé par un espace.

Voici la syntaxe standard utilisée par le système crontab :

Champ	Valeurs autorisées	Rôle
Minute	0 - 59	Définit la minute précise du lancement.
Heure	0 - 23	Définit l'heure (format 24h).
Jour du mois	1 - 31	Définit le numéro du jour dans le mois.
Mois	1 - 12 (ou JAN-DEC)	Définit le mois de l'année.
Jour de la semaine	0 - 6 (ou SUN-SAT)	Définit le jour (0 est souvent dimanche).

Les opérateurs spéciaux :

Pour plus de flexibilité, on utilise des caractères de contrôle :

- *** (Astérisque) :** Signifie "chaque". Un * dans le champ "Mois" signifie que la tâche s'exécute tous les mois.
- **, (Virgule) :** Permet de lister plusieurs valeurs. 1,15 dans le champ "Jour" signifie le 1er et le 15 du mois.
- **- (Tiret) :** Définit une plage de valeurs. 9-17 dans le champ "Heure" signifie toutes les heures de 9h à 17h.
- **/ (Slash) :** Définit un intervalle ou une répétition. */15 dans le champ "Minute" signifie "toutes les 15 minutes".

Quelques Exemples :

0 5 * * *	Tous les jours à 05h00 pile
*/10 * * * *	Toutes les 10 minutes, tous les jours.
0 22 * * 1-5	À 22h00, du lundi au vendredi uniquement

ANNEXE SUR LES ERREURS

Il y a deux types d'erreur possibles :

- Les erreurs Client
- Les erreurs Serveur

Les erreurs Client correspondent à la série des 400 :

- 400 — Bad Request (Requête Incorrecte) : La requête est malformée. Le serveur ne peut pas la comprendre. Cela arrive quand il y a un problème de syntaxe dans l'URL.

- 401 — Unauthorized (Non autorisé) : L'accès nécessite une authentification, mais celle-ci est manquante ou invalide. Cela arrive quand la clé API est absente des headers ou qu'elle est mal mise au niveau de l'URL.

- 402 — Payment Required (Paiement requis) : Code réservé pour une utilisation future, mais utilisé par certaines API pour signaler un abonnement expiré. Cela arrive quand le quota gratuit sur une API payante est épuisé.

- 403 — Forbidden (Interdit) : L'API reconnait l'utilisateur mais ne lui donne pas l'accès à une ressource spécifique. Cela arrive lorsque l'on essaie d'accéder à des ressources dont on n'a pas les droits de lecture

- 404 — Not Found (Non trouvé) : La ressource demandée n'existe pas sur ce serveur. Cela arrive lorsqu'un endpoint est renommé ou que les données que l'on souhaite accéder n'existent plus.

- 422 — Unprocessable Entity (Entité non traitable) : La requête est bien formée et authentifiée, mais le serveur ne peut pas traiter les instructions qu'elle contient (erreur sémantique). Cela arrive lorsque dans le JSON par exemple, le format est bon, mais que la donnée n'est pas exploitable.

Les erreurs Serveur correspondent à la série des 500 :

- 500 — Internal Server Error (Erreur interne du serveur) : Le serveur a rencontré une condition inattendue qui l'a empêché de répondre à la requête. Cela peut arriver quand il y a un bug dans le code Python directement

- 503 — Service Unavailable (Service indisponible) : Le serveur est actuellement incapable de traiter la requête pour cause de surcharge temporaire ou de maintenance. Cela arrive quand trop de requêtes sont demandées au serveur.

TABLE DES ILLUSTRATIONS

Figure 1 : Organigramme de Logical Conseils 5
Figure 2 : Schéma du Projet Open Data 11
Figure 3: Capture d'écran de Postman lors d'une réponse à une requête HTTP 13
Figure 4: Première partie du code : récupération des données + début du traitement 14
Figure 5: Dernière partie : Fin du traitement et rangement dans les dossiers en fonction de l'année 14
Figure 6: L'arborescence des fichiers avec dedans l'un des fichiers JSON retournés et au bon format .. 15
Figure 7: Première partie où on configure le bucket et on importe la bibliothèque Google Cloud 16
Figure 8 : Seconde partie où on modifiera le chemin afin de mettre le résultat dans le bucket 16
Figure 9 : Répertoire de la branche feature/test-streamlit et son arborescence de fichiers 17
Figure 10: Les différentes branches 18
Figure 11: Structure du Git et du projet Open Data 20
Figure 12: Interface d'Artifact Registry, ici on peut y voir les dépôts, leur zone et également le moyen d'en créer via la croix + ... 21
Figure 13: Ici on peut voir la manière dont on créer le dépôt, en y déposant un nom et en décidant de la zone du serveur, plus le serveur est proche de nous géographiquement, moins les coûts seront onéreux. 22
Figure 14: Voici la liste des buckets 22
Figure 15: Interface de création d'un Bucket 23
Figure 16: Exemple de code de routers : commodity_by_type_latest.py pour les matières premières en fonction de leur type 24
Figure 17: Code lié à commodity_by_type_latest.py afin de pouvoir télécharger sous un unique fichier Json, la somme de plusieurs Json, lors de l'appel 25
Figure 18 : Ici on importe les différentes routes 26
Figure 19: Ici on initialise le swagger 27
Figure 20: Interface GCP, où on a les informations du service 27
Figure 21: Interface GCP, où il faudra mettre à jour le Dockerfile du service 28
Figure 22: Interface du Swagger 28
Figure 23: Test de l'API Commodity (Matières Premières) afin d'avoir les données sur le cours de l'argent 29
Figure 24: La réponse de l'API lorsque c'est un succès nous renvoie un fichier téléchargeable Download file 29
Figure 25: Résultat sur la valeur de l'argent en fonction d'une période d'un mois 30
Figure 26: Code pour importer le css 31
Figure 27: Code CSS pour une Sidebar 31
Figure 28: Dossier de la partie Streamlit 31
Figure 29: Début du code Streamlit avec l'importation de toutes les APIs 32
Figure 30: Partie des variables liées à chaque API 33
Figure 31: Partie configuration des pages de l'application 33
Figure 32: Statut des APIs 34
Figure 33: Code de la page d'accueil 35
Figure 34: Version Graphique de la page d'accueil 35
Figure 35: Page d'accueil avec CSS 36
Figure 36: Code de la sélection des choix pour l'API des Évènements 36
Figure 37: Choix de l'Évènement 37
Figure 38: Choix du Sport 37
Figure 39: Choix de la période 38
Figure 40: Résultat de la recherche 38
Figure 41: Résultat de la recherche avec le CSS 39
Figure 42: Vue des données sous le format JSON 39

Bibliographie

Microsoft Azure. (s.d.). *Microsoft Azure Qu'est ce qu'un Data Lake*. Récupéré sur Microsoft Azure : https://azure.microsoft.com/fr-fr/resources/cloud-computing-dictionary/what-is-a-data-lake#:~:text=Définition%20d%27un%20lac%20de%20données,-Ce%20guide%20d&text=Mais%20tout%20d%27abord%2C%20définissons,base%20pour%20divers%20besoins%20analytiques.

Oracle France. (s.d.). *Oracle France Data Warehouse : Qu'est-ce que c'est ?* Récupéré sur Oracle France: https://www.oracle.com/fr/database/data-warehouse-definition/#:~:text=Un%20Data%20Warehouse%20est%20une%20base%20de%20données%20relationnelle%20hébergée,processus%20de%20prise%20de%20décision.

RGPD. (s.d.). *Article 38*. Récupéré sur Réglement Général de la Protection des Données: https://www.cnil.fr/fr/reglement-europeen-protection-donnees/chapitre4#Article38

RGPD. (s.d.). *Article37*. Récupéré sur Réglement Général de la Protection des Données: https://www.cnil.fr/fr/reglement-europeen-protection-donnees/chapitre4#Article37

RGPD. (s.d.). *Article39*. Récupéré sur RGPD: https://www.cnil.fr/fr/reglement-europeen-protection-donnees/chapitre4#Article39

RGPD. (s.d.). *Article5-1-a*. Récupéré sur Réglement Général de la Protection des Données: https://www.cnil.fr/fr/reglement-europeen-protection-donnees/chapitre2#Article5

RGPD. (s.d.). *Article5-1-b*. Récupéré sur Réglement Général de la Protection des Données: https://www.cnil.fr/fr/reglement-europeen-protection-donnees/chapitre2#Article5

RGPD. (s.d.). *Article5-1-c*. Récupéré sur Réglement Général de la Protection des Données: https://www.cnil.fr/fr/reglement-europeen-protection-donnees/chapitre2#Article5

RGPD. (s.d.). *Article5-1-d*. Récupéré sur Réglement Général de la Protection des Données: https://www.cnil.fr/fr/reglement-europeen-protection-donnees/chapitre2#Article5

RGPD. (s.d.). *Article5-1-e*. Récupéré sur Réglement Général de la Protection des Données: https://www.cnil.fr/fr/reglement-europeen-protection-donnees/chapitre2#Article5

RGPD. (s.d.). *Article5-1-f*. Récupéré sur Réglement Général de la Protection des Données: https://www.cnil.fr/fr/reglement-europeen-protection-donnees/chapitre2#Article5

RGPD. (s.d.). *Article83*. Récupéré sur Réglement Général de la Protection des Données: https://www.cnil.fr/fr/reglement-europeen-protection-donnees/chapitre8#Article83

Vertex AI. (s.d.). *Vertex AI*. Récupéré sur Vertex AI: https://investharass.com/?gclid={gclid}&campaign_id={campaignid}&adposition={adposition}&placement={placement}&utm_term={utm_term}&loc_physical={loc_physical}&creative={creative}&utm_source={utm_source}&utm_geo={utm_geo}&sb={sb}&gclid={gclid}&utm_source=g